

Szegedi Tudományegyetem Informatikai Intézet

SZAKDOLGOZAT

Kanton Roderik

2026

Szegedi Tudományegyetem Informatikai Intézet

Kalóriaszámláló alkalmazás.

Szakdolgozat

Készítette:

Kanton

Roderik

programtervező
informatikus
szakos
hallgató

Témavezető:

Dr. Bilicki Vilmos

egyetemi adjunktus

Szeged
2026

1 Feladatkíírás

A modern életmód mellett egyre fontosabbá válik az egészségtudatosság, amelynek egyik alappillére a megfelelő táplálkozás és a napi kalóriabevitel nyomon követése. Sok felhasználó számára nehézséget okoz az élelmiszerek manuális rögzítése és a tápanyagértékek folyamatos számolása.

A feladat egy olyan Android alapú mobilalkalmazás készítése Java nyelven, amely segít a felhasználóknak a napi táplálkozásuk és vízfogyasztásuk menedzselésében. Az alkalmazásnak lehetőséget kell biztosítania a felhasználók regisztrációjára, személyes profil létrehozására (BMR alapú kalkulációkkal), valamint a napi étkezések és fizikai aktivitások (mozgás) egyszerű rögzítése. A bevétel megkönnyítése érdekében integrálni kell egy vonalkód-olvasót (Open Food Facts API használatával) [\[3\]](#), valamint a modern technológiák jegyében egy AI alapú étel-azonosító modult (Gemini AI) [\[3\]](#). Az adatok perzisztens tárolásához a Google Firebase [\[2\]](#) felhőalapú szolgáltatásait kell igénybe venni. A fejlesztés során kiemelt figyelmet kell fordítani a következőkre: - Felhasználóbarát, reszponzív felület kialakítása (Material Design) [\[6\]](#). - Biztonságos autentikáció (Firebase Auth [\[2\]](#)). - Valós idejű adatszinkronizáció (Cloud Firestore) [\[2\]](#). - Külső API-k stabil kezelése (Retrofit) [\[4\]](#). - Mesterséges intelligencia alkalmazása a felhasználói élmény javítására.

2 Tartalmi összefoglaló

2.1 Téma megnevezése:

Fitness Tracker: Táplálkozás- és életmód-követő mobilalkalmazás.

2.2 A megadott feladat megfogalmazása:

Egy olyan natív Android alkalmazás fejlesztése, amely komplex megoldást nyújt a napi kalória- és makrótápanyag-bevitel (szénhidrát, fehérje, zsír) követésére, segítve ezzel a felhasználókat egészségügyi céljaik elérésében.

2.3 Megoldási mód:

Az alkalmazást Java nyelven, Android Studio^[1] környezetben fejlesztettem. Az adatok szinkronizálásához és a felhasználók azonosításához a Firebase^[2]-t használtam. Az élelmiszeradatok lekéréséhez a Retrofit^[4] könyvtárat és az Open Food Facts API^[3]-t, míg az intelligens funkciókhoz a Google Gemini AI^[5] modelljét integráltam.

2.4 Alkalmazott eszközök, módszerek:

Android Studio^[1], Git. - Backend: Firebase Authentication^[2] (email és Google login), Cloud Firestore^[2] (dokumentum alapú NoSQL). - API-k: Retrofit 2^[4], Open Food Facts API^[3], Google AI studio Gemini^[5]. - UI: XML Layouts, Material Design 3^[6], View Binding, ObjectAnimator. - Tesztelés és CI/CD: JUnit^[11], Mockito^[12] (unit tesztek), Espresso^[13] (UI tesztek), Jacoco^[18], GitHub Actions^[14].

2.5 Elért eredmények:

Létrejött egy stabil, reszponzív Android alkalmazás, amely képes a felhasználók napi táplálkozási és mozgási statisztikáinak vezetésére. A felhasználók manuálisan, vonalkód alapján vagy mesterséges intelligencia segítségével is rögzíthetnek étkezési és mozgási adatokat (MET becsléssel). Az alkalmazás automatikusan számolja a BMR értéket és segít a napi célok meghatározásában. A tesztelési fázis igazolta a rendszer megbízhatóságát és az AI modul pontosságát.

2.6 Kulcsszavak:

Android, Java, Firebase, Gemini AI, Táplálkozás követés, Fitness, Open Food Facts, Material Design.

3 Tartalomjegyzék

1	Feladatkiírás	1
2	Tartalmi összefoglaló	2
2.1	Téma megnevezése:.....	2
2.2	A megadott feladat megfogalmazása:.....	2
2.3	Megoldási mód:.....	2
2.4	Alkalmazott eszközök, módszerek:.....	2
2.5	Elért eredmények:.....	2
2.6	Kulcsszavak:	2
3	Tartalomjegyzék	3
4	Bevezetés	7
5	Piackutatás.....	7
5.1	MyFitnessPal(Under Armour) ^[15]	8
5.2	YAZIO (Fast Track) ^[16]	9
5.3	KalóriaBázis ^[17]	9
5.4	Piaci rés és a “Fitness Tracker” küldetése	10
6	Funkcionális specifikáció	10
6.1	Részletes funkcionális követelmények.....	12
6.1.1		12
6.1.2	Személyre szabott élettani profil és célkitűzés	12
6.1.3	Intelligens ételmiszer- és mozgásnaplózási modul.....	12
6.1.4	. Napi Dashboard és Analitika	13
6.2	Nem funkcionális követelmények és minőségi jellemzők	14
7	Tervezett megjelenés	14
7.1	Bejelentkezés és regisztráció.....	14
7.2	Dashboard (Főoldal)	15
7.3	Ételmiszer és mozgásrögzítő felületek	17

7.4	Napi Napló (History)	18
7.5	Statisztikák és grafikonok.....	19
7.6	Felhasználói profil és beállítások.....	20
7.7	Vizuális visszacsatolás (Feedback) rendszer	21
8	Felhasznált technológiák	21
8.1	Android platform és Java	21
8.2	Firestore ökoszisztéma	22
8.3	Hálózati kommunikáció és API-k.....	22
8.4	UI és Design (Material 3 és UX alapelvek).....	22
8.4.1	Vizuális hierarchia és dinamikus megjelenés	23
8.4.2	Komponens architektúra és reszponzivitás	23
8.4.3	Felhasználói élmény (UX) megoldások.....	24
8.4.4	Lokalizáció és Többnyelvűség (i18n)	24
8.5	Tesztelési Technológiák	24
8.6	Egyéb segédkönyvtárak.....	25
8.7	CI/CD Tesztautomatizáció.....	25
8.8	Mesterséges Intelligencia (Gemini AI)	25
9	Architektúra és Rendszerterv	26
9.1	Csomagszerkezet (Package Structure).....	26
9.2	Adatmodell (Modell réteg).....	26
9.3	Üzleti logika és Helper osztályok.....	27
9.4	Firestore adatbázis séma (NoSQL).....	27
9.5	Backend és API architektúra.....	28
9.6	Frontend és Navigációs Terv.....	28
9.7	Dokumentáció, mint Kód (Docs-as-Code).....	29
10	Megvalósítás és Belső Felépítése	29
10.1	Regisztráció és Autentikáció Implementálása	29
10.2	Profilkezelés és BMR számítási logika.....	30
10.3	Napi napló (Daily Entry) és CRUD műveletek.....	30
10.4	Élelmiszer adatbázis és Vonalkód olvasás	30
10.5	AI alapú étel azonosítás implementálása.....	31

10.6	Többnyelvűség és AI prompt-feldolgozás	31
10.7	Statisztikák és Grafikonok vizualizációja	32
10.8		32
10.9	Mozgásnapló és dinamikus kalóriamérleg.....	32
11	Tesztelés	32
11.1	Egységtesztek (Unit tesztek)	33
11.2	Integrációs tesztek	33
11.3	Interface tesztek (UI tests)	34
11.4	Felhasználói elfogadási tesztek (UAT)	34
12	Adatmodell és Entitások.....	34
12.1	Felhasználói profil (User)	35
12.2	Élelmiszer bejegyzés (FoodEntry)	35
12.3	Napi összesítő (DailyLog)	35
12.4	Víznapló (WaterLog)	36
12.5	Mozgás Bejegyzés (Exercise)	36
12.6	Adatkapcsolatok és NoSQL struktúra	36
12.7	JSON reprezentáció és Dokumentum alapú tárolás	37
13	A rendszer magasszintű folyamatai	37
13.1	Adatfelvétel folyamata (Szekvencia-diagram leírás).....	38
13.2	Az adatok mentése és szinkronizációja.....	39
13.3	Profilfrissítés és dinamikus újra számítás.....	39
14	Fontosabb Kódrészletek	40
14.1	A BMR alapú kalóriaszámítás logikája.....	40
14.2	Gemini AI prompt és válasz-feldolgozás	40
14.3	Reaktív adatfrissítés Firestore segítségével	41
15	Tapasztalatok, továbbfejlesztési lehetőségek.....	42
16	Mesterséges Intelligencia Használata a Fejlesztés Során.....	44
17	Összefoglalás	46
18	Irodalomjegyzék	47
19	Nyilatkozat	49
20	Köszönetnyilvánítás	50

Kalóriaszámláló alkalmazás.

21	Elektronikus mellékletek	51
22	Mellékletek.....	52

4 Bevezetés

A 21. században az elhízás és az ehhez kapcsolódó életmódbetegségek globális problémává váltak. Az egészségmegőrzés egyik legfontosabb eszköze a tudatosság: annak ismerete, hogy szervezetünknek mennyi energiára van szüksége, és azt milyen forrásokból visszük be. A digitális forradalom lehetőséget ad arra, hogy ez a követés ne teher, hanem egy egyszerű, rutinszerű mozdulat legyen a mobiltelefonunkon.

Szakedolgozatom célja egy olyan Android alkalmazás kifejlesztése volt, amely túllép a hagyományos kalóriaszámlálókon. A “Fitness Tracker” projekt fókuszában az egyszerűség és az intelligencia áll. Sokan azért hagyják abba a naplózást, mert a termékek kézi keresése és beírása időigényes. Erre a problémára kínál megoldást a vonalkód-olvasás és a legújabb generatív mesterséges intelligencia (Gemini AI) ^[5] bevezetése.

A fejlesztés során törekedtem a tiszta kód elveinek (Clean Code) ^[10] betartására és egy olyan skálázható architektúra kialakítására, amely később könnyen bővíthető további funkciókkal (pl. edzésnapló, közösségi modul). A backend technológiaként választott Firebase ^[2] platform lehetővé teszi a felhasználók számára, hogy adataikat bármilyen eszközről elérjék, miközben a fejlesztő számára leveszi a szerverüzemeltetés terhet.

5 Piackutatás

A szakedolgozatom elkészítésének korai szakaszában átfogó piackutatást végeztem a jelenleg elérhető legnépszerűbb kalória- és életmód-követő alkalmazások körében. A kutatás célja az volt, hogy feltérképezzem a piaci réseket, inspirációt merítsek a felhasználói felületekhez, és azonosítsam azokat a funkciókat, amelyeket a felhasználók hiányolnak vagy amelyekért fizetniük kell a konkurens megoldásoknál.

Funkcionalitás	MyFitnessPal	YAZIO	KalóriaBázis	Fitness Tracker (Saját)
Ingyenes vonalkód-olvasó	FALSE	TRUE	TRUE	TRUE
AI alapú ételfelismerés	FALSE	FALSE	FALSE	TRUE
AI alapú mozgásnapló	FALSE	FALSE	FALSE	TRUE
Magyar nyelvű adatbázis	TRUE	TRUE	TRUE	TRUE
Modern Material 3 design	FALSE	TRUE	FALSE	TRUE
Felhő alapú szinkronizáció	TRUE	TRUE	TRUE	TRUE
BMR alapú kalkuláció	TRUE	TRUE	TRUE	TRUE

1.1. ábra: Összefoglaló táblázat az áttekintett alkalmazásokról.

Az 1.1. ábrán látható táblázat a területi áttekintés során készült összefoglaló táblázat, amely az áttekintett alkalmazások funkcionalitását foglalja össze. A *TRUE* jelzi, ha a alkalmazásban megtalálható az adott funkcionalitás, a *FALSE* jelzi, ha nem található meg az adott funkcionalitás.

5.1 MyFitnessPal(Under Armour) [\[15\]](#)

A MyFitnessPal jelenleg a világ egyik legelterjedtebb kalóriaszámlálója [\[15\]](#), amelynek legnagyobb előnye a több mint 14 millió elemet tartalmazó élelmiszer-adatbázis [\[15\]](#) és a hatalmas, évtizedes múltú visszatekintő közösségi bázis. Részletes elemzés: - Adatbázis mélysége: Az alkalmazás képes kezelni a komplex recepteket is, és szinte minden éttermi lánc menüjét tartalmazza. Ugyanakkor az adatbázis jelentős része felhasználói feltöltésből származik, ami gyakran duplikációkhoz és pontatlan adatokhoz (pl. hiányzó rost- vagy mikro tápanyag adatok) vezet. - Üzleti modell hátrányai: Az alkalmazás az utóbbi években agresszív monetizációs stratégiát követ. A korábban ingyenes vonalkód-olvasó funkciót sok régióban “fizetési fal” (paywall) mögé helyezték. Emellett a kezdőképernyőt nagyméretű hirdetési bannerek foglalják el, ami rontja a felhasználói élményt és lassítja az alkalmazás betöltését régebbi készülékeken. - Technikai hiányosság: Bár rendelkezik gépi tanulási kísérletekkel, a generatív AI (mint a Gemini) integrációja hiányzik, így a felhasználó továbbra is kénytelen kulcsszavas kereséssel vagy manuális bevitelre támaszkodni, ha nem talál vonalkódot.

5.2 **YAZIO (Fast Track)** [\[16\]](#)

A német fejlesztésű YAZIO az esztétikus megjelenésre, a minimalizmusra és a különböző modern diétás trendekre (pl. időszakos böjt - Intermittent Fasting) fókuszál. Részletes elemzés: - UI/UX design: Az alkalmazás kiváló példája a modern Android tervezési irányelveknek. Letisztult kategóriákat és motiváló grafikonokat használ. Ez az esztétika nagyban hozzájárul a felhasználói megtartáshoz (retention), azonban a testreszabhatóság korlátozott az ingyenes verzióban. - Funkcionális korlátok: A YAZIO nagy hangsúlyt fektet az előre elkészített receptcsomagokra, de ezek 90%-a csak prémium előfizetéssel érhető el. A vízfogyasztás követése és a makrótápanyagok részletes (gramm alapú) beállítása is gyakran korlátozott. - Adatkezelés: A nemzetközi adatbázisa erős, de a specifikus, kisebb magyar gyártók termékei esetén gyakran pontatlanabb, mint a hazai fejlesztésű alternatívák.

5.3 **KalóriaBázis** [\[17\]](#)

Magyarország legnagyobb és legrégebbi ételadatbázisával rendelkező platformja. Egyedülálló módon közösségi moderációval tartja karban a magyar élelmiszerek listáját. Részletes elemzés: - Lokalizáció: Ez az egyetlen szoftver, amelyben szinte minden magyarországi bolt (pl. hazai húsboltok, pékségek) saját terméke megtalálható, beleértve a pontos főzési/sütési súlyvesztés-kalkulátorokat is. - Technológiai elavultság: Az Android alkalmazás legnagyobb gyengesége a technikai adósság. A felület nem követi a Google által javasolt Material Design [\[6\]](#) irányelveket (pl. elavult navigációs fiók, nem reszponzív listaelemek). A kódstabilitás gyakran kifogásolható, és a vonalkód-olvasó modul lassabb, mint a modern könyvtárakat (pl. ML Kit) használó megoldások. - Innováció hiánya: A platform konzervatív fejlődési utat jár be, nem integrál modern felhőalapú mesterséges intelligencia megoldásokat a bevitel megkönnyítésére, ami a fiatalabb, technológia-orientált generáció számára hátrányt jelent.

5.4 **Piaci rés és a “Fitness Tracker” küldetése**

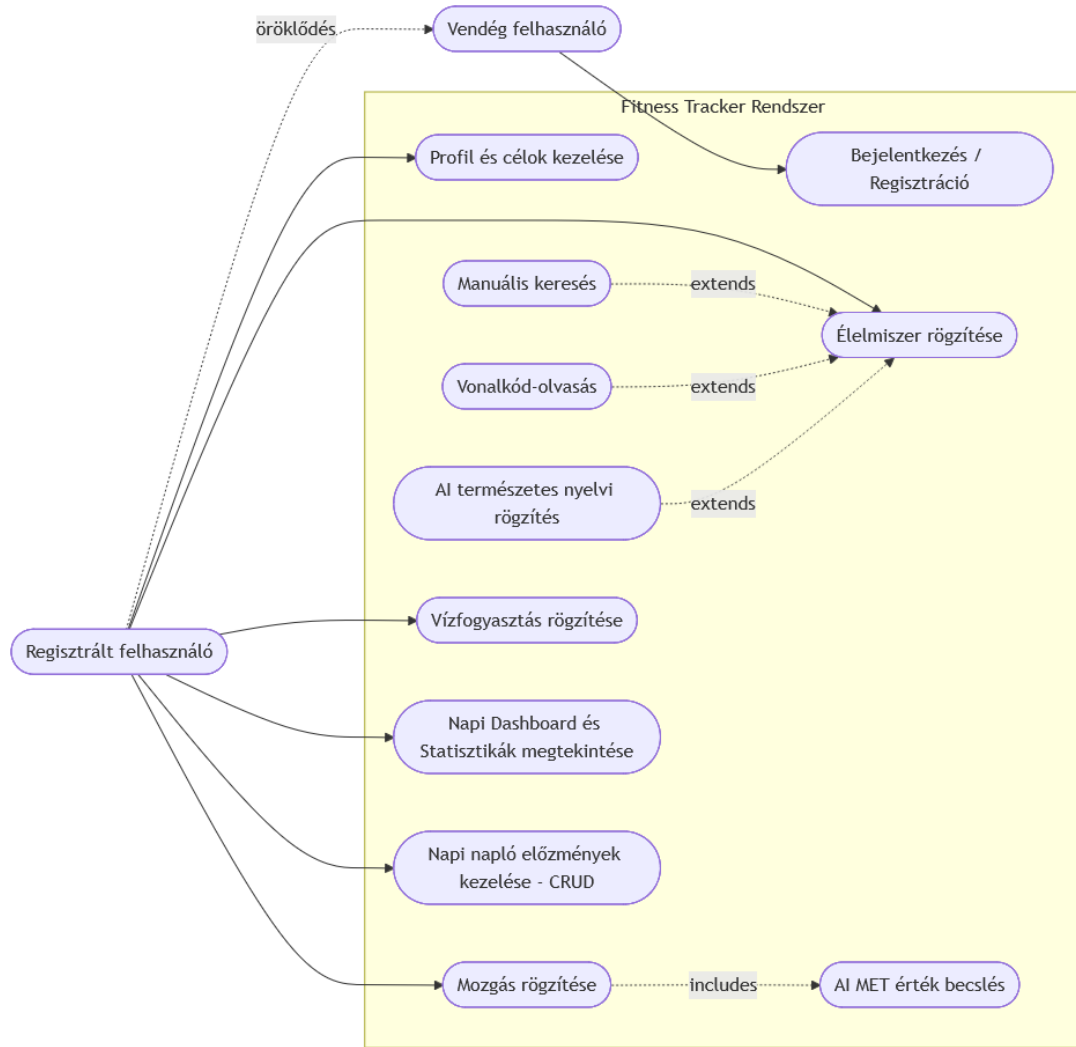
A fenti elemzések alapján azonosítható egy piaci rés: szükség van egy olyan alkalmazásra, amely ötvözi a KalóriaBázis magyar nyelvű pontosságát a YAZIO modern megjelenésével, miközben ingyenesen kínálja a MyFitnessPal által fizetőssé tett funkciókat. A saját fejlesztésű alkalmazásom ezen felül a Gemini AI [\[5\]](#) segítségével egy teljesen új adatbeviteli módot vezet be: a “természetes nyelvi rögzítést”, amely áthidalja a keresőmezők és a vonalkód-leolvasás közötti szakadékot (pl. ha a felhasználó egy étteremben eszik, ahol nincs vonalkód, de le tudja írni az ételt).

Az alkalmazás funkcionális követelményeit úgy határoztam meg, hogy a rendszer ne csupán egy passzív naplózó eszköz legyen, hanem aktív támogatást nyújtson az egészségmegőrzésben. A fejlesztés során kiemelt figyelmet fordítottam az adatbeviteli akadályok lebontására, mivel a piackutatás során ez bizonyult a legfőbb felhasználói lemorzsolódási oknak.

6 **Funkcionális specifikáció**

A használati eset diagram segítségével a rendszerünk által elérhető funkcionálisokat és az azokat használó aktorok közötti kapcsolatokat modellezhetjük le a legmagasabb absztrakciós fokon. Az aktorok a felhasználók, akik valamilyen módon interakcióba lépnek a rendszerrel, míg a használati esetek maguk a rendszer által kínált funkciók. Ezek a diagramok mutatják be a rendszer tervezett funkcióit, a rendszer környezetét és ezek kapcsolatait.

Kalóriaszámláló alkalmazás.



1.2. ábra: A alkalmazáshoz készült használati eset diagram

A használati eset diagramok jelölésrendszere az UML része. Az aktorokat, felhasználókat pálcikaemberrel, a rendszer funkcióit oválissal, az aktorok és funkciók közötti kapcsolatokat különböző vonalakkal jelöljük.

Az alkalmazás felhasználóit két csoportba sorolhatjuk: vendég felhasználók és regisztrált fiókkal rendelkező felhasználók. A két csoport közötti különbség jelentős, hiszen a regisztrált fiókkal rendelkező felhasználók sokkal több funkciót érnek el, mint az oldalra bejelentkezés nélkül látogató felhasználók.

6.1 Részletes funkcionális követelmények

6.1.1

6.1.2 Személyre szabott élettani profil és célkitűzés

Az alkalmazás az első bejelentkezés után bekéri a felhasználó paramétereit, amelyek alapján a rendszer dinamikusan kalibrálja önmagát: - Dinamikus BMR kalkulátor: A program a megadott nem, kor, súly és magasság alapján a Mifflin-St Jeor képletet [\[9\]](#) alkalmazva kiszámítja az alapanyagcserét. Ez az érték szolgál a napi kalóriakeret alapjául. - Célorientált módosítás: A felhasználó három állapot közül választhat: - Fogyás: A BMR érték csökkentése (kalóriadeficit). - Súlytartás: A BMR értéknek megfelelő keret. - Tömegelés: Megnövelt kalória- és fehérjebevitel javaslata. - Profil módosítás: Lehetőség van a testsúly periodikus frissítésére, ami után a rendszer újraszámolja a napi kereteket, így követve a felhasználó haladását.

6.1.3 Intelligens ételmisszer- és mozgásnaplózási modul

Ez az alkalmazás legösszetettebb része, amely négy redundáns módszert kínál az adatok rögzítésére:

1. Vonalkód-alapú azonosítás (Scanner):

- A kamera interfész segítségével a szoftver valós időben keres EAN-13 szabványú kódokat. - Sikeres felismerés esetén a Open Food Facts API-n [\[3\]](#) keresztül lekéri a termék nevét, márkáját és 100g-ra vetített tápértékeit (energia, zsír, szénhidrát, fehérje).
 - Amennyiben a termék nem található, a rendszer felajánlja a manuális rögzítés lehetőségét, ezzel is bővítve a felhasználó saját helyi adatbázisát.
- #### **2. Természetes nyelvi feldolgozás (AI Log):**
- A felhasználó szabadszövegesen megadhatja, mit evett (pl.: “Reggelire ettem két rántottát és egy szelet barna kenyert”).

Kalóriaszámláló alkalmazás.

- A Gemini 2.5 Flash modell [\[5\]](#) elemzi a szöveget, szétválasztja az összetevőket, és megbecsüli azok tömegét és tápértékét.
- A visszakapott adatokat a felhasználó jóváhagyhatja vagy módosíthatja a véglegesítés előtt.

3. Hagyományos kereső és előzmények:

- Kulcsszavas keresés a Firebase-ben [\[2\]](#) tárolt globális és saját élelmiszerlistában.
- A rendszer megjegyzi a leggyakrabban fogyasztott ételeket, így azok egyetlen kattintással újra hozzáadhatók a naplóhoz.

4. Tevékenységkövetés (Mozgás rögzítése):

- A felhasználó rögzítheti a fizikai aktivitását, ahol a Gemini AI a megadott mozgásforma alapján MET (Metabolic Equivalent of Task) [\[9\]](#) becslést végez, és ez alapján dinamikusan számolja az elégetett kalóriát.

6.1.4 . Napi Dashboard és Analitika

A főképernyő feladata a gyors információátadás a Material 3 [\[6\]](#) tervezési elvek mentén: - Kalória-mérleg: Megmutatja a napi keretet, az eddig elfogyasztott mennyiséget, az edzéssel elégetett energiát és a még hátralévő kalóriákat (dinamikus kalóriamérleg). - Makrótápanyag vizualizáció: Kördiagram vagy sávdiagram segítségével szemlélteti a fehérje, szénhidrát és zsír arányát, segítve a kiegyensúlyozott táplálkozást. - Hidratációs modul: Egy dedikált számláló a vízfogyasztásnak, ahol előre beállított egységekkel (2dl, 5dl) gyorsan rögzíthető a bevitel. - Mozgásnapló (Exercise Log): A fizikai aktivitás rögzítése, ahol a mesterséges intelligencia becsüli meg az elégetett kalóriát. - Idővonal: Az adott napon elfogyasztott ételek listája, ahol lehetőség van elemek törlésére vagy módosítására (CRUD műveletek).

6.2 Nem funkcionális követelmények és minőségi jellemzők

- Platformstabilitás: Az alkalmazásnak minimum Android 8.0 (Oreo) verziótól hibamentesen kell futnia.
- Adatintegritás: A Cloud Firestore szinkronizációnak [\[2\]](#) köszönhetően az adatok nem veszhetnek el az alkalmazás bezárásakor vagy az eszköz újraindításakor.
- Válaszidő: Az AI alapú feldolgozás nem tarthat tovább 5 másodpercnél. A vonalkódos keresésnek (hálózati sebességtől függően) 2 másodpercen belül eredményt kell adnia.
- Ergonómia (UX): A Material You design [\[6\]](#) nyelv használatával a felületnek alkalmazkodnia kell a modern Android eszközök vizuális világához (pl. kerekített sarkok, tiszta tipográfia).
- Offline működés: Az alkalmazásnak alapvető funkcióiban (pl. korábban rögzített adatok megtekintése) hálózati kapcsolat nélkül is működőképesnek kell maradnia (helyi cache használatával).
- Nemzetköziesítés (i18n): Az alkalmazásnak hozzáférhetőnek kell lennie a nemzetközi piacon is, minimálisra csökkentve a nyelvi korlátokat a kulcsfunkciók (például kalóriaszámlálás, profilbeállítás) használata során.

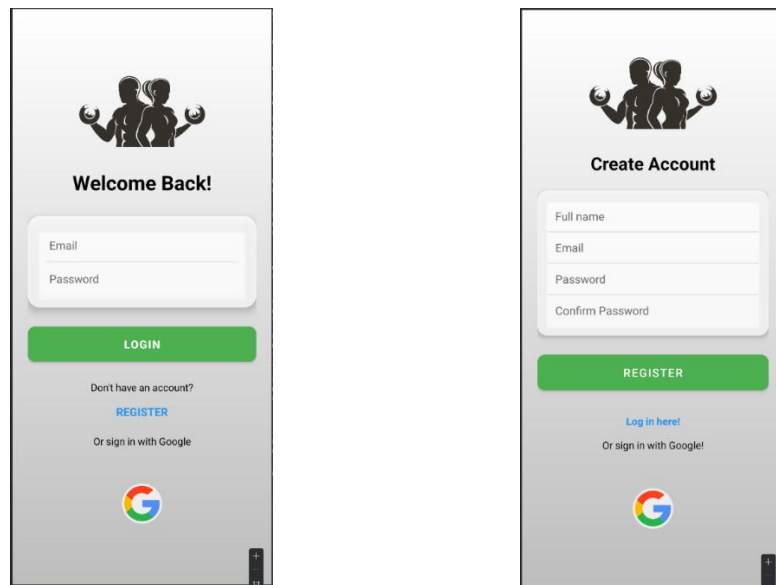
7 Tervezett megjelenés

Az alkalmazás vizuális terveit és a felhasználói élmény (UX) alapjait a modern Android fejlesztési elvek, különösen a Material Design 3 (Material You) [\[6\]](#) figyelembevételével alakítottam ki. A tervezésnél a fő szempont az volt, hogy a felhasználó a lehető legkevesebb interakcióval (ún. “friction”) rögzíthesse az adatait, miközben folyamatos visszacsatolást kap az egészségügyi állapotáról.

7.1 Bejelentkezés és regisztráció

Az alkalmazás indításakor a felhasználó a belépő felülettel találkozik. Mivel az adatok tárolása a Cloud Firestore-ban [\[2\]](#) történik, a hitelesítés elengedhetetlen. • Funkció: Email és jelszó alapú autentikáció (Firebase Authentication) [\[2\]](#). • UX megoldás: A

regisztráció során az app több lépcsőben (Step-by-step) kéri be az élettani adatokat (nem, kor, súly, magasság), hogy ne riassza el a felhasználót egyetlen hosszú űrlappal. • Navigáció: Sikeres belépés után a rendszer a Dashboardra navigál, míg elfelejtett jelszó esetén egy helyreállító felületre juthat a felhasználó



1.2.1. ábra: Az alkalmazás bejelentkezés és regisztrációs felülete világos megjelenése

7.2 Dashboard (Főoldal)

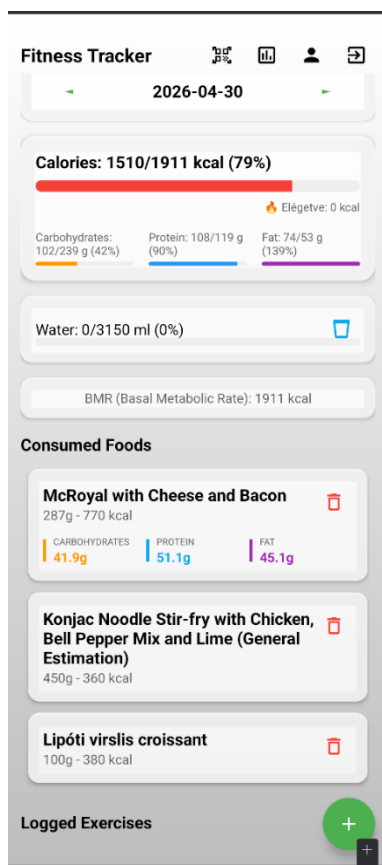
Ez az alkalmazás központi egysége, ahol a felhasználó egyetlen pillantással felmérheti a napi haladását.

- Mire való: Itt jelenik meg a napi kalóriamérleg (keret - elfogyasztott + elégetett), valamint a makrótápanyagok (fehérje, szénhidrát, zsír) megoszlása.
- Hogy jutsz oda: Ez az alapértelmezett kezdőképnyő a belépés után.

Kalóriaszámláló alkalmazás.

- Interakciók: A képernyőn található egy “Floating Action Button” (FAB), amely a gyors rögzítési menüt nyitja meg. Innen indítható a vonalkód-olvasó vagy a manuális keresés is.

1.2.2. ábra: Az alkalmazás belépő felületének világos megjelenése



A bejelentkezést, illetve regisztrációt a Firebase Authentication^[2] szolgáltatás segítségével valósítottam meg.

7.3 Élelmiszer és mozgásrögzítő felületek

A piackutatás alapján négy különálló felületet terveztem az adatok bevitelére:

1. Manuális kereső: Egy listázó nézet, ahol kulcsszavas kereséssel (pl. “csirkemell”) érhető el a lokális és globális adatbázis.
2. Vonalkód-olvasó: A kamera interfészét használó nézet, amely felismeri a termékeket. Ha a termék ismert, azonnal felugrik egy adatlap a tápértékekkel.
3. AI (Gemini) Napló: Egy szövegbeviteli mező, ahová a felhasználó természetes nyelven írhat. Az AI által feldolgozott adatok egy “ellenőrző” képernyőn jelennek meg, ahol a felhasználó finom hangolhatja a mennyiségeket a végleges mentés előtt, akkor jelenik meg a gomb, ha nem talál az adatbázisban egyező inputot.
4. Mozgás rögzítő (Add Exercise): Egy dedikált felület, ahol a felhasználó a végzett mozgás típusát és időtartamát rögzítheti, amely alapján az AI megbecsüli a kalóriaégetést.

The image displays two mobile application screens side-by-side. The left screen, titled 'New Exercise', features a back arrow, tabs for 'FOOD' and 'EXERCISE' (the latter is active), a text input for 'Type of exercise:' with the example 'e.g. Running', a numeric input for 'Duration (min):' set to '0' with a 'min' unit, a purple link 'Calculate calories with AI', and a green 'ADD EXERCISE' button at the bottom. The right screen, titled 'Add New Item', also has a back arrow and tabs for 'FOOD' and 'EXERCISE' (the latter is active). It includes a 'Pick a Food' dropdown menu showing 'Sült Csirkeszárny', a 'Quantity (grams):' input field with the placeholder 'Enter the quantity', a unit selector set to 'gram', and a green 'ADD FOOD' button at the bottom.

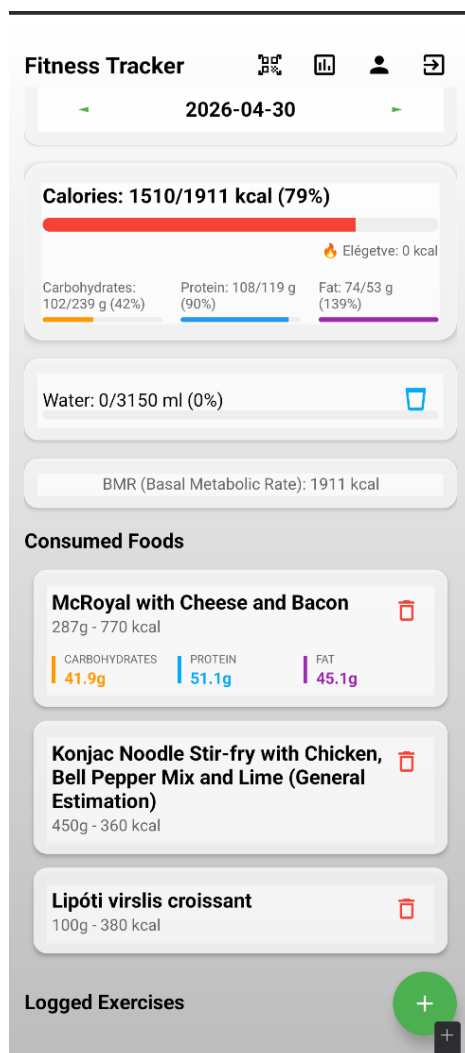
1.2.3.

ábra: Az étel hozzáadása oldal világos megjelenése

1.2.4.

ábra: Az edzés hozzáadása oldal világos megjelenése

7.4 Napi Napló (History)

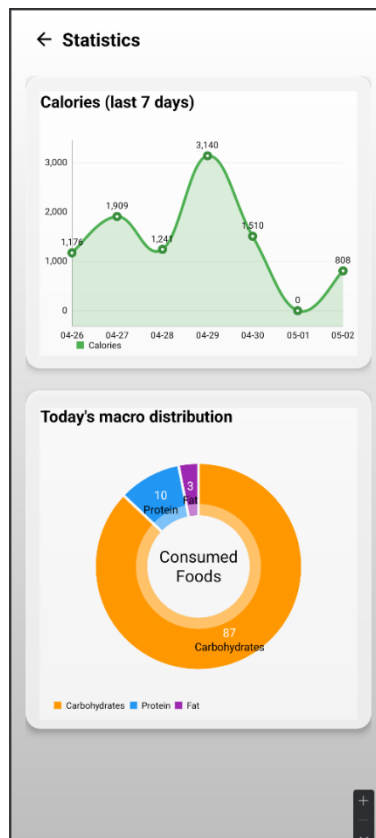


1.2.5. ábra: Jegyzet részletes oldal világos megjelenéssel

Ez a felület az időrendi sorrendben elfogyasztott ételeket listázza.

- Mire jó: Visszakövethető, hogy melyik étkezés mennyi kalóriát jelentett, és itt van lehetőség a hibás bejegyzések törlésére vagy módosítására (CRUD műveletek).
- UX: A listaelemekre kattintva a felhasználó újra megtekintheti az adott étel részletes tápértékeit (pl. mennyi telített zsír vagy rost volt benne).

7.5 Statisztikák és grafikonok

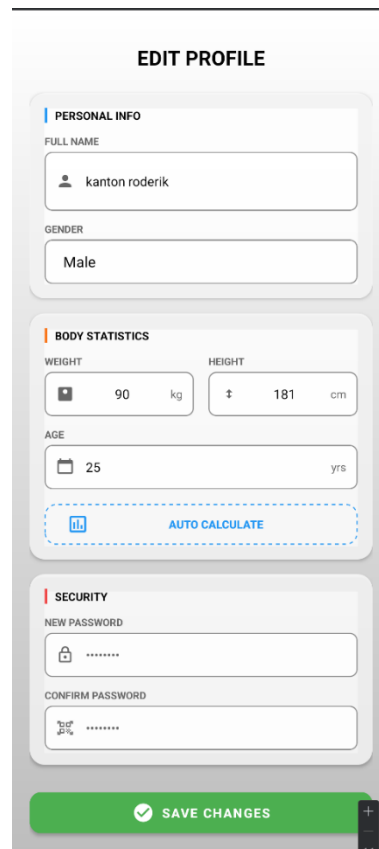
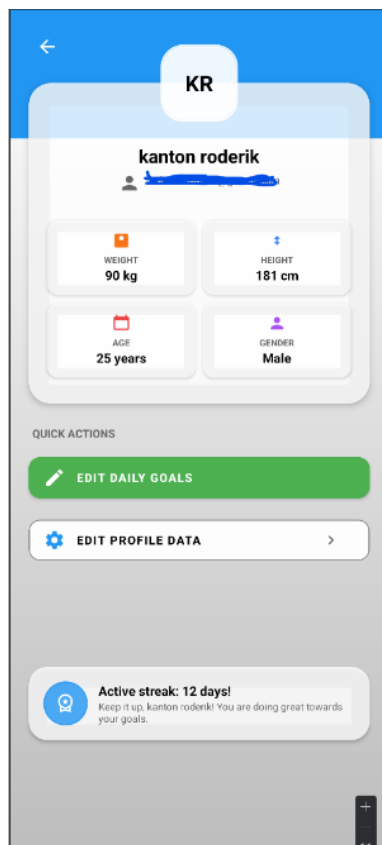


3.5.1. ábra: Statisztika részletes oldal világos megjelenéssel

A hosszú távú motivációt segítő felület, amely az utolsó hét adatait összesíti.

- Mire való: Itt vizualizálom a súlyváltozást és az átlagos kalóriabevitelt grafikonok (LineChart) segítségével.
- Hova tudsz menni: Ha egy adott nap grafikonjára kattint a felhasználó, a rendszer visszavigálja őt az adott napi részletes naplóhoz

7.6 Felhasználói profil és beállítások



- 1.2.6. ábra: Profil részletes oldal világos megjelenéssel
1.2.7. ábra: Beállítások oldal világos megjelenéssel

Itt történik a személyes paraméterek és célkitűzések kezelése. • Mire való: A felhasználó itt módosíthatja a célját (pl. fogyásról tömegelésre váltás). Ennek hatására az app minden kalkulációt (BMR, napi keret) azonnal frissít. • Hogy jutsz oda: A felső sávban (AppBar) található profil ikonnal vagy az alsó menüsor végén. • Hova tudsz menni: Innen érhető el a kijelentkezés, illetve az alkalmazás technikai adatai (verziószám, adatvédelmi nyilatkozat).

7.7 Vizuális visszacsatolás (Feedback) rendszer

A modern UX jegyében minden folyamatnak van látható jele:

- Töltési fázis: Mivel a Firebase és a Gemini API is hálózati kommunikációt igényel, minden lekérés közben egy Material Shimmer (csillogó effektus) vagy egy CircularProgressIndicator [\[6\]](#) látható, így a felhasználó nem érzi azt, hogy az app lefagyott.
- Hibüzenetek: Ha a vonalkód nem található az Open Food Facts adatbázisában, egy Snackbar üzenet tájékoztatja a felhasználót, és felkínálja a manuális rögzítés lehetőségét.

8 Felhasznált technológiák

Az alkalmazás fejlesztése során a modern Android ökoszisztéma szabványos és megbízható eszközeit alkalmaztam, szem előtt tartva a skálázhatóságot, a biztonságot és a gyors fejlesztési ciklust.

8.1 Android platform és Java

Az alkalmazás natív Android környezetben készült [\[1\]](#). A választott programozási nyelv a Java, amely stabil, objektumorientált alapokat nyújt az Android keretrendszerrel való interakcióhoz.

- Választás indoklása: A Java nyelv széleskörű könyvtártámogatása és a dokumentációk mélysége tette lehetővé a komplex API-integrációk (mint a Gemini vagy a Retrofit) stabil megvalósítását.

- Minimum SDK: Az alkalmazás az API 26-os szintet (Android 8.0 Oreo) célozza meg, ami biztosítja a modern funkciók használatát a piacon lévő eszközök nagy részén, miközben stabil futási környezetet nyújt.

8.2 Firebase ökoszisztéma

A Google Firebase platformját alkalmaztam “Serverless” backendként [\[2\]](#), ami szükségtelemmé tette egy saját szerveroldali infrastruktúra kiépítését.

- Firebase Authentication: Kezeli a felhasználók biztonságos regisztrációját és beléptetését (Email/Jelszó alapú hitelesítés).
- Cloud Firestore: NoSQL alapú, dokumentum-orientált adatbázis a felhasználói profilok és napi naplók valós idejű tárolására és szinkronizálására.

8.3 Hálózati kommunikáció és API-k

Az alkalmazás külső adatforrásokra támaszkodik az élelmiszeradatok hitelessége érdekében. • Retrofit 2: Típusbiztos HTTP kliens [\[4\]](#), amelyet a REST API hívások kezelésére használtam. A válaszok feldolgozását a GSON konverter segítette [\[8\]](#), amely a JSON válaszokat automatikusan Java objektumokká (POJO) alakítja. • Open Food Facts API [\[3\]](#): Nyílt adatbázis, amely a vonalkód-olvasó funkció mögötti több millió élelmiszer adatait szolgáltatja.

8.4 UI és Design (Material 3 és UX alapelvek)

Az alkalmazás vizuális megjelenése és interakciós modellje a Google által fémjelzett Material Design 3 (Material You) irányelveken alapul [\[6\]](#). A tervezés során az elsődleges szempont a kognitív terhelés minimalizálása és az adatok transzparens megjelenítése volt.

8.4.1 **Vizuális hierarchia és dinamikus megjelenés**

- Dinamikus színek (Dynamic Color): Az alkalmazás támogatja a Material You motorját, így a felület színpalettája képes alkalmazkodni a felhasználó rendszerszintű témájához.
- Tipográfia és kontraszt: A hierarchia egyértelműsítése érdekében eltérő betűsúlyokat és méreteket alkalmaztam (pl. a kalóriaszámláló kiemelt Headline stílust kapott).

8.4.2 **Komponens architektúra és reszponzivitás**

- ConstraintLayout [\[1\]](#): A komplex nézetek esetében ezt alkalmaztam, ami lehetővé teszi a rugalmas elrendezést különböző kijelző méreteken, elkerülve a teljesítményromlást.
- RecyclerView és ViewHolder [\[1\]](#) minta: A napi étkezési napló listázásához használt technológia, amely biztosítja a gördülékeny (60+ FPS) élményt nagy adatmennyiség esetén is.
- Material Components: [\[6\]](#) Olyan elemeket integráltam, mint a Floating Action Button (FAB) a gyors rögzítéshez és a Navigation Bar az egyszerű navigációhoz.

8.4.3 Felhasználói élmény (UX) megoldások

- Vizuális visszacsatolás: Az AI-alapú feldolgozás során LinearProgressIndicator és Shimmer effektek [\[6\]](#) tájékoztatják a felhasználót a folyamatban lévő műveletekről.
- Interakciók: Az alkalmazás figyelembe veszi az egykezes használat elvét; a legfontosabb gombok a képernyő alsó harmadában helyezkednek el.

8.4.4 Lokalizáció és Többnyelvűség (i18n)

A felület nemcsak a kijelző méretekhez, hanem a felhasználó nyelvi beállításaihoz is dinamikusan alkalmazkodik. A szoftveres megvalósítás alapja az Android Resource (strings.xml) alapú lokalizációs mechanizmusa [\[11\]](#). A felület feliratainak és üzeneteinek elkülönítésével külön erőforrásfájlokat hoztam létre a támogatott nyelvekhez (HU, EN, FR, DE). A rendszer a készülék Locale beállítása alapján automatikusan a megfelelő nyelvi csomagot tölti be, így biztosítva a nemzetközi hozzáférhetőséget.

8.5 Tesztelési Technológiák

A szoftver megbízhatóságát és a kódminőséget dedikált tesztelési keretrendszerekkel biztosítottam:

- JUnit 4 [\[11\]](#): Az üzleti logika (például a kalóriaszámítási algoritmusok és BMR képletek) egységtesztelésére (Unit Testing) használtam.
- Espresso [\[13\]](#): UI-tesztelő keretrendszer, amellyel automatizáltam a felhasználói felület interakcióit (pl. bejelentkezési folyamat ellenőrzése).
- Mockito [\[12\]](#): Mockoló keretrendszer, amely lehetővé tette a külső függőségek (Firebase, API) szimulálását az izolált tesztelés érdekében.

8.6 Egyéb segédkönyvtárak

Google ML Kit (Barcode Scanning) [\[7\]](#): A vonalkódok optikai felismeréséért és dekódolásáért felelős nyílt forráskódú könyvtár.

8.7 CI/CD Tesztautomatizáció

A kódminőség és a stabilitás garantálására modern tesztautomatizációs és CI/CD megoldásokat vezettem be a projekt során:

- GitHub Actions [\[14\]](#): CI/CD pipeline az automatizált build, tesztelés és kódminőség ellenőrzésére minden módosításkor.
- JaCoCo [\[18\]](#): Kódlefedettség (Code Coverage) riportok generálása az egységtesztekhez.
- Dependabot [\[14\]](#): A projekt függőségeinek automatikus, biztonsági célú monitorozása és frissítése.

8.8 Mesterséges Intelligencia (Gemini AI)

Az alkalmazás intelligens funkcióinak magját a Google generatív mesterséges intelligenciája, a Gemini 2.5 Flash modell adja [\[5\]](#).

- Természetes nyelvi feldolgozás (NLP): Lehetővé teszi, hogy a felhasználók kötetlen szöveggént rögzítsék az étkezéseiket és a fizikai aktivitásukat, melyet az AI strukturált JSON adattá alakít a perzisztens tárolás és a további adatfeldolgozás céljából.
- Költséghatékony és gyors működés: A Flash modell a Gemini család sebességre és alacsony válaszidőre optimalizált változata, így mobilalkalmazásban valós idejű interakciókhoz is kiválóan használható.
- Google AI studio API: A közvetlen kapcsolat biztosítja az AI modell natív,

biztonságos és hatékony elérését az Android alkalmazásból.

9 Architektúra és Rendszerterv

Az alkalmazás tervezésekor a moduláris és tiszta architektúra kialakítására törekedtem, hogy az egyes rétegek (adatkezelés, üzleti logika, megjelenítés) elválaszthatóak és tesztelhetők legyenek. A rendszer alapját a modern Android fejlesztésben elterjedt alapelvek és a Firebase felhőalapú szolgáltatásai alkotják.

9.1 Csomagszerkezet (Package Structure)

A projekt logikai egységek mentén szerveződik, ami segíti az átláthatóságot és a kód karbantarthatóságát:

- `com.fitness.tracker.activities/.fragments`: A felhasználói felület (UI) vezérlői, a nézetek közötti navigáció és az életciklus-kezelés helye.

- `com.fitness.tracker.models`: A rendszerben használt adatobjektumok (POJO) definíciói.
- `com.fitness.tracker.network`: A külső API kommunikációért (Retrofit, Open Food Facts) felelős interfészek és kliensek.
- `com.fitness.tracker.repository`: Az adatforrások (Firestore és API-k) és a UI közötti közvetítő réteg [\[1\]](#).
- `com.fitness.tracker.utils`: Statikus helper osztályok (pl. számítási algoritmusok, dátumformázók).

9.2 Adatmodell (Modell réteg)

Az adatmodell a rendszer gerince, amely meghatározza, hogyan ábrázoljuk az entitásokat Java objektumként és a Firestore-ban:

- User: Tartalmazza a felhasználó alapvető fiziológiai adatait (nem, kor, súly, magasság, célkitűzés) és a számított BMR értékeket.
- FoodEntry: Egy konkrét étkezést reprezentál (név, kalória, makrotápanyagok, időpont).
- Exercise: Egy konkrét mozgást reprezentál (név, elégetett kalória, kategória, időtartama, MET érték).
- DailyLog: Egy adott naphoz tartozó összes étkezést, mozgást és vízfogyasztást összefogó aggregátor objektum.

9.3 Üzleti logika és Helper osztályok

Az alkalmazás “agya”, ahol a nyers adatokból információ lesz:

- NutritionCalculator: Ebben az osztályban implementáltam a Mifflin-St Jeor képletet [\[9\]](#). Ez a helper osztály felelős a napi kalóriaszükséglet dinamikus újra számolásáért, ha a felhasználó frissíti a súlyát vagy módosítja a célját.
- GeminiParser: Egy speciális logikai egység, amely a Gemini AI-től érkező válaszokat (string/JSON) alakítja át a rendszer számára értelmezhető FoodEntry objektumokká. Az architektúrát a későbbiekben továbbfejlesztettem, és a FirestoreRepository osztályban bevezettem a Dependency Injection (DI) mintát, amely elengedhetetlen volt a tesztelhetőség és a mockolhatóság (Mockito) érdekében.

9.4 Firestore adatbázis séma (NoSQL)

Mivel a Firebase Cloud Firestore egy dokumentum-alapú NoSQL adatbázis [\[2\]](#), a sémát a gyors lekérdezhetőségre optimalizáltam:

- Users kollekció: A dokumentum azonosítója a Firebase Auth által generált egyedi uid. Ez biztosítja a kapcsolatot a hitelesített felhasználó és az adatai között.

- Entries (Alkollekció): Minden felhasználó dokumentuma alatt található egy `daily_entries` kollekció. Itt a dokumentumok nevei dátum alapúak (pl. 2024-04-27), ami lehetővé teszi, hogy egyetlen lekérdezéssel megkapjuk az adott nap minden adatát.

9.5 Backend és API architektúra

A backend szolgáltatások két fő pilléren nyugszanak:

- Firebase Services [\[2\]](#): A hitelesítésért, a perzisztens adattárolásért és a felhőalapú AI (Gemini AI) eléréséért felelős komplex ökoszisztéma.
- REST API Gateway: A Retrofit kliensen [\[4\]](#) keresztül érjük el a külső Open Food Facts adatbázist. A kommunikáció aszinkron módon, külön háttérszálon történik, hogy ne blokkolja a felhasználói felületet.

9.6 Frontend és Navigációs Terv

A megjelenítési réteg az Activity-Fragment modellt követi:

- Single Activity Architecture: Az alkalmazás egy fő MainActivity-re épül, amelyben a különböző funkciókat (Dashboard, Log, Profile) Fragment-ek valósítják meg.
- View Binding: A kódstabilitás érdekében View Binding-ot használtam, elkerülve a lassabb és hibalehetőségeket hordozó `findViewById` hívásokat [\[1\]](#).
- State Management: Az adatok változása esetén a UI automatikusan frissül a Firebase Listenerek segítségével [\[2\]](#), biztosítva a folyamatos, “real-time” élményt.

9.7 Dokumentáció, mint Kód (Docs-as-Code)

A szoftverarchitektúra és a tervezési folyamat dokumentálására a Docs-as-Code megközelítést alkalmaztam. Markdown formátumban, közvetlenül a verziókezelőben tárolva készültek el az Architecture Decision Record (ADR) fájlok (amelyek rögzítik a DI, az állapotkezelés és a hibakezelés bevezetését), a C4 diagramok, a STRIDE alapú fenyegetés-modellezés (Threat Model), valamint a fejlesztői üzemeltetési kézikönyvek (Runbooks).

10 Megvalósítás és Belső Felépítése

Az alkalmazás belső szerkezetét úgy alakítottam ki, hogy az Android keretrendszer életciklus-kezelési sajátosságait ötvözze a Firebase aszinkron adatszolgáltatásaival. A megvalósítás során a kód modularitására és az egyes funkciók logikai elkülönítésére törekedtem.

10.1 Regisztráció és Autentikáció Implementálása

A beléptetési folyamat a LoginActivity és RegisterActivity osztályokban valósul meg.

- Logika: A Firebase Authentication SDK-t használva [\[2\]](#) a hitelesítés nem a helyi eszközön, hanem a Google biztonságos szerverein történik.
- Belső folyamat: Regisztrációkor a rendszer először validálja az adatokat (pl. email formátum, jelszó hossza), majd sikeres válasz esetén létrehozza a felhasználó egyedi rekordját a Firestore users kollekciójában is.

10.2 Profilkezelés és BMR számítási logika

Ez a rész felelős a felhasználó élettani adatainak karbantartásáért.

- Helper osztály: Létrehoztam egy NutritionMath osztályt, amely statikus metódusként tartalmazza a Mifflin-St Jeor képletet [\[9\]](#). Ez biztosítja, hogy a számítási logika központosított legyen, és a UI-tól függetlenül tesztelhető maradjon.
- Folyamat: Amint a felhasználó módosítja a súlyát a ProfileFragment-en, egy trigger fut le, amely újraszámolja a napi kalóriakeretet, és azonnal frissíti a Firestore dokumentumot.

10.3 Napi napló (Daily Entry) és CRUD műveletek

A napló a DailyLogFragment felületén jelenik meg, amely az alkalmazás leggyakrabban használt része.

- Komponens: A megjelenítéshez RecyclerView-t és egy egyedi LogAdapter osztályt használtam.
- Adatkezelés: Megvalósítottam a teljes CRUD (Create, Read, Update, Delete) ciklust. A felhasználó nemcsak hozzáadhat, de hosszan nyomva (Long Press) törölhet is elemeket a naplóból. A változások a SnapshotListener segítségével azonnal, újraindítás nélkül frissülnek a képernyőn.

10.4 Élelmiszer adatbázis és Vonalkód olvasás

A vonalkód-olvasó technikai megvalósításához a Google ML Kit (Barcode Scanning) könyvtárat integráltam [\[7\]](#).

- Szolgáltatás (Service): Egy BarcodeScanner osztály kezeli a kamera interfészt. Sikeres felismerés után a Retrofit [\[4\]](#) interfész segítségével hívom meg az Open Food Facts API-t [\[3\]](#).
- Aszinkronitás: Mivel a hálózati hívás blokkolná a UI-t, a kérést egy háttérszálon (Background Thread) futtatom, miközben a felhasználó egy ProgressBar-t lát.

10.5 AI alapú étel azonosítás implementálása

A projekt egyik leginnovatívabb része a FoodFragment és ExerciseFragment osztály, amelyek a Google AI studio Gemini API-t használják [\[5\]](#).

- Prompt Engineering: A háttérben az alkalmazás nemcsak a felhasználó szövegét küldi el, hanem egy előre definiált "System Instruction" -t is, amely arra utasítja az AI-t, hogy a választ szigorúan JSON formátumban (név, kalória, makrók) adja vissza.
- Parsing: A válaszként kapott stringet a GSON könyvtár [\[8\]](#) alakítja át Java objektummá, mielőtt az bekerülne az adatbázisba.

10.6 Többnyelvűség és AI prompt-feldolgozás

Az alkalmazás többnyelvűségét (HU, EN, FR, DE) nemcsak a statikus UI elemek (strings.xml), hanem az AI prompt-szintű felkészítése is támogatja, ez pedig egy egyedi technikai kihívást támasztott. A kulcskérdés az volt, hogy az AI (Gemini) [\[5\]](#) milyen nyelven kapja a kontextust és a bemenetet, illetve milyen nyelven válaszoljon. Mivel a felület és a generált JSON kulcsok struktúrája angol nyelvű, a rendszer utasításában (System Instruction) elvártam, hogy a válasz formátuma kötött maradjon, de maga a mesterséges intelligencia képes legyen felismerni és elemezni a különböző nyelveken bevitt ételneveket vagy fizikai aktivitásokat. Így a felhasználó a saját nyelvén rögzítheti az adatokat, a feldolgozás pedig teljesen nyelvfüggetlen marad, zökkenőmentes nemzetközi élményt biztosítva.

10.7 Statisztikák és Grafikonok vizualizációja

Az adatok elemzését a StatisticsFragment végzi.

- Adatfeldolgozás: A szoftver lekéri az utolsó 7 nap DailyLog bejegyzéseit a Firestore-ból, és egy Map<String, Integer> adatszerkezetbe rendezi őket dátum szerint.
- Megjelenítés: A vizualizációhoz egy külső grafikon rajzoló könyvtárat használtam, amely dinamikusan skálázza az Y-tengelyt a felhasználó kalóriabevitelének függvényében.

10.8

10.9 Mozgásnapló és dinamikus kalóriamérleg

A projekt kései fázisában integráltam az ExerciseFragment-et a kalóriadeficit pontosabb számítása érdekében. A felhasználó által megadott mozgásforma (pl. '30 perc futás') alapján a rendszer meghívja a Gemini AI-t [\[5\]](#), amely egy MET értéket ad vissza. Ezt a testsúllyal és az időtartammal megszorozva a rendszer dinamikusan kiszámítja az elégetett kalóriákat, majd a Firestore szinkronizáció révén azonnal módosítja (megnöveli) a még elfogyasztható napi kalóriakeretet a Dashboardon.

11 Tesztelés

A szoftverfejlesztési életciklus elengedhetetlen része a minőségbiztosítás. A Fitness Tracker alkalmazás tesztelése során többszintű megközelítést alkalmaztam, a legkisebb logikai egységektől a teljes felhasználói folyamatokig, biztosítva a rendszer stabilitását és az adatok pontosságát. A tesztek futtatását nemcsak manuálisan végeztem, hanem a GitHub Actions segítségével [\[14\]](#) egy CI/CD folyamatba integráltam, így minden kódmódosításnál automatikusan lefutott a build és az ellenőrzés.

11.1 Egységtesztek (Unit tesztek)

Az egységtesztelés célja az alkalmazás üzleti logikájának ellenőrzése izolált környezetben. Ehhez a JUnit 4 keretrendszert használtam [\[11\]](#). A tesztek lefedettségét a Jacoco plugin segítségével mértem [\[18\]](#), amely vizuális HTML riportokban mutatja meg a kódbázis tesztelt részeit.

- BMR kalkuláció tesztelése: A NutritionMath osztályban implementált Mifflin-St Jeor képlet [\[9\]](#) helyességét különböző profilokra (férfi/nő, különböző súlycsoportok) teszteltem. Ellenőriztem, hogy a szélsőséges bemeneti értékek (pl. 0 kg vagy negatív kor) esetén a rendszer megfelelő hibakezelést vagy alapértelmezett értékeket alkalmaz-e.
- Dátum- és adatformázók: Teszteltem a napi naplóhoz használt dátumkezelő logikát, biztosítva, hogy az adatbázis-lekérdezések kulcsai (pl. 2024-04-27) minden időzónában konzisztensek maradjanak.

11.2 Integrációs tesztek

Ebben a fázisban az alkalmazás és a külső szolgáltatások közötti adatforgalmat vizsgáltam, a Mockito keretrendszer segítségével [\[12\]](#) szimulálva a hálózati válaszokat.

- Firebase és Offline szinkronizáció: Megvizsgáltam, hogy az alkalmazás megfelelően kezeli-e a hálózati kapcsolat megszakadását. Ellenőriztem, hogy a Firestore helyi gyorsítótárazása (cache) biztosítja-e a folyamatos adatelérést, és a kapcsolat helyreálltakor az adatok megfelelően szinkronizálódnak-e.
- API Response Mapping: Teszteltem a Retrofit kliens működését az Open Food Facts API-val. Olyan teszteseteket hoztam létre, ahol az API hiányos vagy hibás JSON-t küld vissza, biztosítva, hogy a GSON konverter ne okozzon alkalmazásösszeomlást (Crash), hanem értelmezhető hibaüzenetet adjon a felhasználónak.

11.3 Interface tesztek (UI tests)

A felhasználói élmény folytonosságát az Espresso keretrendszerrel automatizáltam [\[13\]](#), amely lehetővé teszi a felhasználói interakciók szimulálását valós eszközön vagy emulátoron.

- Login Flow: Automatikus tesztet futtattam a bejelentkezési folyamatra, ellenőrizve, hogy helyes adatokkal a Dashboardra, helytelen adatokkal pedig a megfelelő hibaüzenetet megjelenítő Snackbar-ra navigál-e a rendszer.
- Navigációs tesztek: Ellenőriztem a BottomNavigationView működését, biztosítva, hogy a Fragment-ek közötti váltás során az adatok nem vesznek el, és az UI állapota megmarad (pl. képernyőelforgatás után is).

11.4 Felhasználói elfogadási tesztek (UAT)

A technikai teszteken túl manuális, “fekete doboz” tesztelést is végeztem egy szűk tesztcsoport bevonásával.

- Módszer: A tesztelőknek konkrét célokat kellett elérniük (pl. “Rögzíts egy 500 kcal-s ebédet az AI segítségével”).
- Eredmény: A visszajelzések alapján finomítottam a vonalkód-olvasó érzékenységén és javítottam a Gemini AI által generált válaszok megjelenítési sebességét egy informatívabb töltőképernyő (Progress Indicator) bevezetésével.

12 Adatmodell és Entitások

Az alkalmazás adatmodelljét a Cloud Firestore NoSQL adatbázis sajátosságaihoz igazítottam [\[2\]](#). A relációs adatbázisokkal ellentétben itt nem táblák és sorok, hanem dokumentumok és kollekciók határozzák meg a struktúrát. Ez a felépítés lehetővé teszi a rugalmas sémakezelést és a gyors, skálázható lekérdezéseket.

12.1 Felhasználói profil (User)

A felhasználói entitás az alkalmazás alapköve, amely a Firebase Authentication egyedi azonosítójához (UID) kapcsolódik.

- Mezők: uid (String), name (String), age (Int), gender (String), height (Float), weight (Float), activityLevel (String).
- Származtatott adatok: Itt tárolódik a számított bmrValue és a választott targetCalories is.
- Szerepe: Ez az objektum határozza meg a számítások alapját, és minden egyéb adat (étkezés, víz) ehhez a profilhoz van rendelve.

12.2 Élelmiszer bejegyzés (FoodEntry)

Ez az entitás reprezentál egy konkrét elfogyasztott élelmiszert vagy ételt.

- Mezők: id (String), foodName (String), calories (Double), protein (Double), carbs (Double), fat (Double), servingSize (Double), timestamp (Long).
- Forrás: A bejegyzés létrejöhet manuális bevitelből, Open Food Facts API válaszból vagy a Gemini AI által generált adatokból.

12.3 Napi összesítő (DailyLog)

Napi összesítő (DailyLog) A hatékony lekérdezhetőség érdekében az adatokat napok szerint csoportosítottam.

- Szerkezet: Egy dokumentum képvisel egy napot (pl. 2024-04-27 névvel).
- Tartalom: Tartalmazza a totalCalories, totalMacros (Map) és az exercisesDone (Map) mezőket, amelyek az adott napi bejegyzések összesített értékeit mutatják, elkerülve, hogy minden megnyitáskor újra kelljen számolni az összes elemet (denormalizáció).

12.4 Víznapló (WaterLog)

A folyadékbevitel követése külön egyszerűsített entitást kapott a gyorsabb írási műveletek érdekében.

- Mezők: date (String), amount (Int - milliliterben), goal (Int).
- Funkció: Lehetővé teszi a napi vízfogyasztás inkrementális növelését egyetlen gombnyomással.

12.5 Mozgás Bejegyzés (Exercise)

A fizikai aktivitásokat reprezentáló entitás, amely az elégetett kalóriák nyomon követésére szolgál.

- Mezők: id (String), name (String), duration (Int - percekben), caloriesBurned (Double), timestamp (Long).
- Funkció: A rögzített mozgás után a kiszámolt elégetett kalóriák hozzáadódnak a felhasználó napi elfogyasztható kalóriakeretéhez.

12.6 Adatkapcsolatok és NoSQL struktúra

A rendszerben a “Subcollection” modellt alkalmaztam [\[2\]](#). Minden egyes User dokumentum alatt található egy DailyEntries alkollekció.

- Előnye: Ez a struktúra biztosítja a legmagasabb szintű adatbiztonságot. A Firebase Security Rules segítségével könnyen definiálható, hogy egy felhasználó csak a saját alkollekcióihoz férhessen hozzá (/users/{userId}/DailyEntries/{entryId}).

12.7 JSON reprezentáció és Dokumentum alapú tárolás

Mivel a Firestore JSON-szerű formátumban tárolja az adatokat, az objektumok leképezése (Mapping) közvetlenül történik a Java POJO osztályok és a tárolt dokumentumok között. • Példa egy FoodEntry JSON-re:

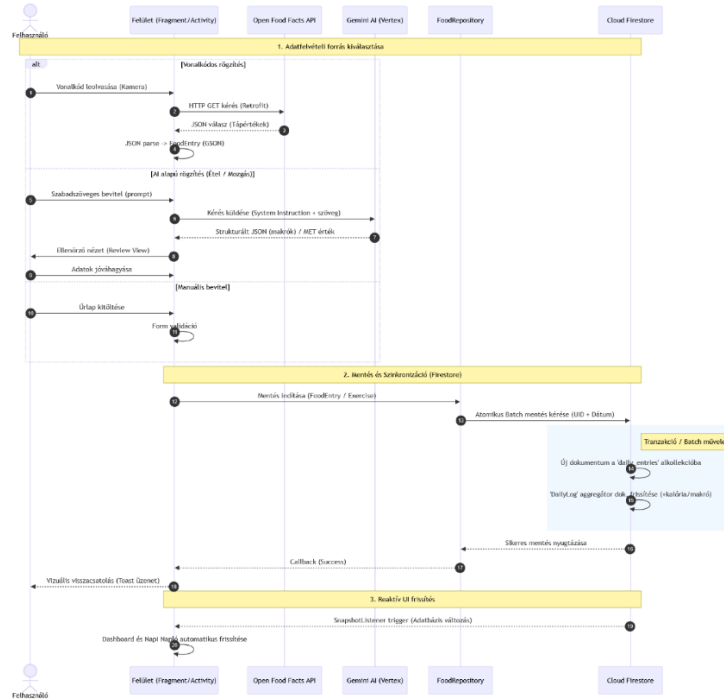
```
{  
  "foodName": "Csirkemell rizzsel",  
  "calories": 450,  
  "protein": 35.5,  
  "carbs": 45.0,  
  "fat": 8.5,  
  "timestamp": 1714231200000  
}
```

Ez a formátum lehetővé teszi, hogy az alkalmazás minimális adatforgalommal és transzformációs költséggel kommunikáljon a felhővel.

13 A rendszer magasszintű folyamatai

Az alkalmazás magasszintű folyamatai közül a legösszetettebbek az élelmiszer-rögzítési munkafolyamatok. Ebben a fejezetben bemutatom az adatok útját a felhasználói interakciótól a felhőalapú tárolásig, különös tekintettel az aszinkron műveletekre módosítása.

13.1 Adatfelvétel folyamata (Szekvencia-diagram leírás)



9.1. ábra: A megevett étel felvételének a szekvencia diagramja

Az ételrögzítés folyamata három különböző forrásból indulhat, de minden esetben a FoodRepository osztályban fut össze.

1. Vonalkódos rögzítés: A folyamat a BarcodeScannerActivity-vel indul. Sikeres leolvasás után a rendszer nem közvetlenül ment, hanem a Retrofit segítségével [4] meghívja az Open Food Facts API-t [3]. Az API válaszaként kapott JSON-t a GSON konverter [8] alakítja FoodEntry objektummá.
2. AI-alapú rögzítés: A felhasználó szabadszöveges bevitel után a GeminiService elküldi a kérést a Google felhőbe [5]. A válasz feldolgozása után egy ellenőrző nézet (Review View) jelenik meg, ahol a felhasználó jóváhagyhatja az adatokat.
3. Manuális bevitel: Itt a folyamat lineáris: az űrlap validálása után az adatok közvetlenül a mentési fázisba kerülnek.

4. Mozgás (Exercise) rögzítése: A felhasználó megadja a végzett mozgásformát és az időtartamot. A rendszer a háttérben meghívja a Gemini AI-t a mozgásformához tartozó MET (Metabolic Equivalent) érték megbecsléséhez. A testsúly és a MET érték alapján az alkalmazás kiszámítja az elégetett kalóriákat, majd a létrehozott Exercise objektum a mentési fázisba kerül.

13.2 Az adatok mentése és szinkronizációja

Amint a FoodEntry objektum rendelkezésre áll, a FirestoreService veszi át az irányítást [\[2\]](#). A mentési folyamat az alábbi lépésekből áll:

- Dokumentum azonosítása: A rendszer meghatározza az aktuális felhasználó UID-ját és az aktuális dátumot.
- Atomikus művelet: A rendszer elmenti az új étkezést a daily_entries kollekcióba, és ezzel párhuzamosan frissíti a napi összesítő dokumentumot (DailyLog), hogy a Dashboard azonnal a helyes értékeket mutassa.
- Vizuális visszacsatolás: A sikeres mentést egy Toast üzenet és a UI automatikus frissítése (a Firestore SnapshotListener segítségével) jelzi a felhasználónak.

13.3 Profilfrissítés és dinamikus újra számítás

A rendszer egyik kulcsfolyamata a BMR értékek naprakészen tartása.

- Amikor a felhasználó a Profil oldalon módosítja a súlyát, a ProfileFragment meghívja a NutritionMath.calculateBMR() függvényt, amely a Mifflin–St Jeor képletet alkalmazza [\[9\]](#).
- Az új értékek nem csak az adatbázisba kerülnek be, hanem a rendszer azonnal frissíti a memóriában lévő User objektumot is, így a Dashboard-ra visszalépve a felhasználó már az új kalóriakeretet látja.

14 Fontosabb Kódrészletek

Ebben a fejezetben bemutatom az alkalmazás azon forráskód-részleteit, amelyek a rendszer üzleti logikájának gerincét alkotják, és amelyek megoldást nyújtanak a feladatkiírásban szereplő komplex kihívásokra.

14.1 A BMR alapú kalóriaszámítás logikája

Az alkalmazás személyre szabott működésének alapja a felhasználó élettani adatai alapján végzett kalkuláció. A NutritionMath osztályban implementáltam a Mifflin-St Jeor képletet [\[9\]](#), amely jelenleg a legpontosabb módszer az alapanyagcsere (BMR) meghatározására

```
7 usages
public static double calculateBMR(double weight, double height, double age, Gender gender) {
    if (weight <= 0 || height <= 0 || age <= 0) return 0;
    double bmr;
    if (gender == Gender.MALE) {
        bmr = (10 * weight) + (6.25 * height) - (5 * age) + 5;
    } else {
        bmr = (10 * weight) + (6.25 * height) - (5 * age) - 161;
    }
    return Math.max(0, bmr);
}
```

Ez a kód biztosítja, hogy minden felhasználó a saját céljainak (fogyás, tömegelés) megfelelő napi keretet kapjon, amely az alapja minden további naplózási funkciónak

14.2 Gemini AI prompt és válasz-feldolgozás

Az alkalmazás egyik leginnovatívabb része az AI-alapú bevitel. A technikai kihívást itt az jelentette, hogy a mesterséges intelligencia válaszát strukturált, a program által feldolgozható formátumba (JSON) kényszerítsem.

```
1 usage
private void setupAi() {
    GenerationConfig.Builder configBuilder = new GenerationConfig.Builder();
    configBuilder.responseMimeType = "application/json";
    GenerationConfig config = configBuilder.build();

    Content systemInstruction = new Content.Builder()
        .addText("Act as a fitness and nutrition expert. Provide nutrition data for 100g and common units." +
            " Return ONLY a raw JSON object. Fields: name, calories, carbs, protein, fat, commonUnits (array).")
        .build();

    GenerativeModel gm = new GenerativeModel(
        modelName: "gemini-1.5-flash",
        buildConfig.GEMINI_API_KEY,
        config,
        safetySettings: null,
        new RequestOptions( timeout: 30000L, apiVersion: "v1beta"),
        tools: null,
        toolConfig: null,
        systemInstruction
    );

    generativeModel = GenerativeModelFutures.from(gm);
}
```

A “System Prompting” alkalmazásával elértem, hogy a modell ne felesleges magyarázó szöveget, hanem egy közvetlenül deszerializálható adatsomagot adjon vissza [\[5\]](#), minimalizálva a hibalehetőséget.

14.3 Reaktív adatfrissítés Firestore segítségével

Annak érdekében, hogy a Dashboard azonnal frissüljön új étel rögzítésekor, a Firebase SnapshotListener-ét alkalmaztam [\[2\]](#). Ez a megoldás feleslegessé teszi a képernyő manuális frissítését vagy az adatok újra lekérését.

Ez a részlet jól szemlélteti a felhőalapú adatbázis előnyeit: az alkalmazás és az adatbázis állapota mindig szinkronban marad, ami kiemelkedő felhasználói élményt nyújt.

```

6 usages
private void startListeningToData() {
    stopListening();
    String selectedDate = getFormattedDate();

    dailyEntryListener = repository.listenToDailyEntry(selectedDate, ( DocumentSnapshot snapshot, FirebaseFirestoreException error) -> {
        if (error != null) {
            Log.e(TAG, msg: "Daily entry listen failed", error);
            return;
        }

        if (snapshot != null) {
            DailyEntry entry = snapshot.exists() ? snapshot.toObject(DailyEntry.class) : new DailyEntry(selectedDate);
            if (entry != null) {
                entry.calculateTotals();
                this.currentEntry = entry;
                foodAdapter.updateData(entry.getConsumedFoods());

                if (entry.getExercisesDone() != null) {
                    Log.d(TAG, msg: "Edzések száma: " + entry.getExercisesDone().size());
                    exerciseAdapter.updateData(entry.getExercisesDone());
                } else {
                    exerciseAdapter.updateData(new HashMap<>());
                }
            }

            updateUI();
        }
    });
}

```

15 Tapasztalatok, továbbfejlesztési lehetőségek

A szakdolgozatom készítése előtt már tisztában voltam azzal, hogy Android alapú mobilalkalmazást fogok fejleszteni Java nyelven, a backend szolgáltatásokhoz pedig a Firebase-t fogom használni. Ennek oka az volt, hogy az Android fejlesztéssel és a Java nyelvvel már korábban is volt tapasztalatom, de szerettem volna egy komplexebb, modern API-kat és architektúrát (például Material Design 3) integráló projektben jobban elmélyülni. A Firebase pedig kézenfekvő választás volt, mivel a szerveroldali infrastruktúra kiépítése nélkül biztosít megbízható autentikációt és valós idejű adatbázist.

A tervezés fázisa előtt kisebb prototípusokat készítettem, hogy leteszteljem a projekt kulcsfontosságú technológiáit. Utánanéztem, hogy az Open Food Facts API milyen formátumban adja vissza a vonalkódos keresések eredményeit, és hogy a Google Gemini AI modellje mennyire megbízhatóan képes természetes nyelvű szövegekből JSON formátumú tápanyag adatokat generálni. Mivel az előzetes tesztek során mindkét technológia jól teljesített, magabiztosan kezdhettem neki a rendszertervezésnek.

Első lépésként létrehoztam a projektet az Android Studio -ban, amelyet azonnal feltöltöttem egy Git tárolóba a folyamatos verziókövetés érdekében. A fejlesztés során igyekeztem a feladatokat kisebb, jól definiálható egységekre bontani.

A fejlesztést a felhasználói felület és a képernyőtervek kialakításával folytattam. A tervezés során a modern Material You (Material Design 3) irányelveit követtem, célom egy letisztult, reszponzív és a felhasználók számára könnyen értelmezhető felület létrehozása volt. Kiemelt figyelmet fordítottam a megfelelő színpaletta és a tipográfia kiválasztására, hogy az alkalmazás megjelenése motiváló, mégis átlátható legyen. Bár a komplex nézetek és animációk (például a betöltést jelző Shimmer effektek) megvalósítása olykor kihívást jelentett, rendkívül izgalmas volt megtapasztalni, hogyan áll össze a vizuális élmény.

A felületi tervek után az adatbázis sémájának kialakítására fókuszáltam. A Cloud Firestore dokumentum-alapú NoSQL struktúrájához igazodva megterveztem a kollekciókat és a dokumentum-hierarchiát, majd definiáltam a szükséges biztonsági szabályokat, hogy a felhasználók csak a saját adataikhoz férhessenek hozzá. Ez a megközelítés gyors, biztonságos és jól skálázható adatkezelést tett lehetővé.

Ezt követően az üzleti logika és az API-k integrálásával foglalkoztam. A fejlesztés ezen szakasza gördülékenyen haladt. Kisebb nehézséget a Retrofit hálózati hívások aszinkron kezelése és a Gemini AI válaszainak hibátűrő feldolgozása okozott, de ezeket a megfelelő kivételkezeléssel és adatmodellezéssel sikeresen megoldottam. A Firebase nagyban megkönnyítette a munkámat, hiszen az Authentication és a Firestore SnapshotListener révén a bejelentkezés és a valós idejű adatszinkronizáció egyszerűen és stabilan implementálható volt.

A projekt végső fázisában a kezdeti tervekhez képest sikeresen implementáltam egy CI/CD csővezetékot (GitHub Actions, Dependabot), és a Docs-as-Code elv mentén átstrukturáltam a teljes dokumentációt. Szintén extra funkcióként valósult meg a mozgásnapló (Exercise Log) és a Gemini AI alapú MET becslés, ami dinamikussá tette a kalóriamérleget. Bár az alkalmazás jelenlegi formájában stabil és funkciógazdag, a jövőben tovább növelhetnék az alkalmazás értékét az alábbiak:

Képfelismerés alapú ételrögzítés: A Gemini AI vizuális képességeit kihasználva a szöveges bevétel mellett lehetőség nyílna az ételek fotó alapján történő azonosítására és a tápértékek automatikus becslésére.

Okos eszköz-szinkronizáció: A Google Health Connect, Garmin vagy más Wearable API-k integrálásával az alkalmazás automatikusan beolvashatná a felhasználó napi lépésszámát és elégetett kalóriáit.

Személyre szabott ajánlások és közösségi funkciók: A meglévő adatok elemzésével proaktív étrendi tanácsok biztosítása, valamint egy közösségi modul bevezetése a motiváció növelésére, ahol a felhasználók recepteket oszthatnának meg, vagy kihívásokban vehetnének részt.

Összességében úgy gondolom, hogy egy olyan natív Android alkalmazást sikerült létrehoznom, amely komplex funkcióinak köszönhetően könnyen és gyorsan használható, valamint a modern felhőalapú és mesterséges intelligencia szolgáltatások révén hatékonyan támogatja a felhasználókat az egészségtudatos életmód kialakításában.

16 Mesterséges Intelligencia Használata a Fejlesztés Során

A modern szoftverfejlesztési életciklus ma már elképzelhetetlen a mesterséges intelligencia által nyújtott támogatás nélkül. A szakdolgozat elkészítése során a mesterséges intelligenciát nem úgy kezeltem, mint ami “megírja helyettem” a projektet, hanem mint egy agilis mérnöki eszközt, amellyel növelhetem a produktívitásomat, és validálhatom a fejlesztési döntéseimet. Ennek keretében tudatosan választottam és alkalmaztam különböző MI alapú technológiákat a projekt eltérő szakaszaiban.

A fejlesztés során három fő MI eszközt használtam. A kódolás mindennapos, mikroszintű támogatására a GitHub Copilot volt a segítségemre, amely az Android Studio [\[1\]](#) környezetbe integrálva valós időben kínált kódkiegészítéseket. Az átfogóbb, architektúrais kérdések megvitatására, bonyolultabb hibakeresésre (debugging) és a dokumentáció formázásának segítésére a ChatGPT (GPT-4o) modellt vettem igénybe. Emellett, mivel a projekt maga is a Google technológiáira és a Gemini AI-ra épül, a

rendszerbe épített “System Instruction” promptok (rendszerutasítások) tesztelésére és optimalizálására a Gemini Advanced webes felületét használtam.

Ezeket az eszközöket számos konkrét részfeladat során alkalmaztam. A Copilotot leginkább a sok ismétlődéssel járó, úgynevezett “boilerplate” kódok generálására használtam: ilyenek voltak például az Android RecyclerView adaptereinek váza, a ViewBinding inicializálási blokkok, vagy a JSON válaszokhoz tartozó POJO (Plain Old Java Object) adatszerkezetek, ahol az MI pillanatok alatt legenerálta a szükséges getter-setter és adattranszformációs metódusokat. A ChatGPT-t egyfajta “senior fejlesztői” partnerként kezeltem. Gyakran alkalmaztam kódreview-ra, megkérve, hogy keressen potenciális memóriaszivárgásokat (memory leak) a Fragment-ek életciklus-kezelésében. Szintén a ChatGPT segített az egységtesztek (JUnit) és mock objektumok felépítésének alapszerkezetében, vagy amikor egy nehezen olvasható hibaüzenetet (stack trace) dobott a Retrofit hálózati réteg, aminek az okát gyorsan szerettem volna felderíteni.

Minden generált eredményt szigorú validációnak vettem alá, mivel az MI válaszait sosem tekintettem abszolút igazságnak, csupán erős hipotézisként kezeltem őket. A munkám során többször találkoztam a modellek “hallucinációjával”. Például, amikor a kamerás vonalkód-olvasóhoz kértem a navigációs kódot, a Copilot és a ChatGPT is a régen elavult, de a betanítási adatokban még domináns startActivityForResult metódust ajánlotta az Android modern ActivityResultLauncher API-ja helyett. Egy másik esetben a ChatGPT egy olyan Firebase Firestore [\[2\]](#) lekérdezést (Query) generált számomra, amely technikailag ugyan szintaktikailag helyes volt, de az összetett where() záradékok miatt egy különálló, összetett index (Composite Index) manuális beállítását igényelte volna a Google Cloud konzolon – egy lépés, amit a modell “elfelejtett” említeni, s ez futásidejű hibához vezetett. Ezeket a hibákat minden esetben a hivatalos dokumentáció (Android Developers [\[1\]](#), Firebase Docs [\[2\]](#)) átolvasásával, valamint izolált futtatásokkal és egységtesztekkel szűrtem ki.

Számos olyan területe volt a projektnek, ahol tudatosan nem használtam az MI segítségét. Az alkalmazás felhasználói felületének tervezését (Material Design 3 irányelvek mentén) és a pontos vizuális hierarchia (XML elrendezések) kialakítását teljes egészében kézzel végeztem. Ennek oka az, hogy az MI a tapasztalataim szerint hajlamos “széteső” vagy elavult layout struktúrákat (pl. a ConstraintLayout helyett régi

RelativeLayout-okat) generálni, és nem érzi azokat az ergonómiai finomságokat, amelyek egy modern mobilalkalmazás sajátjai. Hasonlóképpen nem vettem igénybe az MI-t az alkalmazás központi üzleti logikájának megtervezésekor (például, hogy az adatbázis sémában milyen denormalizációs stratégiát követelek), valamint a jelenlegi és a konklúziót tartalmazó fejezetek gondolati megfogalmazásakor, hiszen ezek a mérnöki döntések a szakmai önállóságot tükrözik.

Összességében a mesterséges intelligencia használata rendkívül tanulságos volt. A legnagyobb pozitívum egyértelműen az implementációs sebesség növekedése volt: a fávágó munkák (kódkiegészítés, adatmodellek írása) idejének drasztikus csökkentése teret engedett a magasabb szintű, architektúra szintű gondolkodásnak. Ugyanakkor lassított, amikor túlzottan bíztam a javaslataiban; egy esetben egy helytelen Gradle konfigurációt generáltatott velem a modell, aminek a visszaállítása és a probléma manuális felderítése órákat vett igénybe. A jövőbeli projektjeim során ezt a tapasztalatot felhasználva az MI-t még szigorúbb keretek között alkalmaznám: sokkal inkább ötvözném a Test-Driven Development (TDD) módszertannal, azaz először definiálnám és megírnám a pontos teszteket, majd ezeken keresztül ellenőrizném a generált kódot, csökkentve ezzel a hibalehetőségeket és a technológiai adósság felhalmozódását.

17 **Összefoglalás**

Szakdolgozatom keretében egy funkcionálisan gazdag, modern Android alkalmazást sikerült fejlesztenem. A Fitness Tracker nem csupán egy kalóriaszámláló, hanem egy olyan intelligens asszisztens, amely a Firebase és a mesterséges intelligencia erejét használja fel az egészségtudatos életmód támogatására. A fejlesztés során mélyebb ismereteket szereztem a NoSQL adatbázis-kezelés, a felhőalapú szolgáltatások és az AI integráció területén. Az alkalmazás tesztelt, stabil és alkalmas a mindennapi használatra.

18 Irodalomjegyzék

1. Google Developers. Android Developers Documentation. Elérhető: <https://developer.android.com/> (Utolsó megtekintés: 2026. 05. 03.)
2. Google. Firebase Documentation (Authentication, Cloud Firestore). Elérhető: <https://firebase.google.com/docs> (Utolsó megtekintés: 2026. 05. 03.)
3. Open Food Facts. Open Food Facts API Documentation. Elérhető: <https://world.openfoodfacts.org/data> (Utolsó megtekintés: 2026. 05. 03.)
4. Square, Inc. Retrofit: A type-safe HTTP client for Android and Java. Elérhető: <https://square.github.io/retrofit/> (Utolsó megtekintés: 2026. 05. 03.)
5. Google. AI studio Gemini API Elérhető: <https://aistudio.google.com/api-keys> (Utolsó megtekintés: 2026. 05. 03.)
6. Google. Material Design 3. Elérhető: <https://m3.material.io/> (Utolsó megtekintés: 2026. 05. 03.)
7. Google ML Kit (Barcode Scanning): <https://developers.google.com/ml-kit/vision/barcode-scanning> (Utolsó megtekintés: 2026. 05. 03.)
8. Google. Gson: A Java serialization/deserialization library to convert Java Objects into JSON and back. Elérhető: <https://github.com/google/gson> (Utolsó megtekintés: 2026. 05. 03.)
9. Mifflin, M. D., St Jeor, S. T., Hill, L. A., Scott, B. J., Daugherty, S. A., & Koh, Y. O. (1990). A new predictive equation for resting energy expenditure in healthy individuals. The American journal of clinical nutrition, 51(2), 241-247.
10. Martin, Robert C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. ISBN: 978-0132350884.
11. JUnit Team. JUnit 4 Framework. Elérhető: <https://junit.org/junit4/> (Utolsó megtekintés: 2026. 05. 03.)
12. Mockito Contributors. Mockito framework site. Elérhető: <https://site.mockito.org/> (Utolsó megtekintés: 2026. 05. 03.)
13. Google Developers. Espresso testing framework. Elérhető: <https://developer.android.com/training/testing/espresso> (Utolsó megtekintés: 2026. 05. 03.)
14. GitHub: <https://github.com/> (Utolsó megtekintés: 2026. 05. 03.)
15. MyFitnessPal: <https://www.myfitnesspal.com/> (Utolsó megtekintés: 2025. 09. 30.)
16. YAZIO: <https://www.yazio.com/en> (Utolsó megtekintés: 2025. 09. 30.)

Kalóriaszámláló alkalmazás.

17. KalóriaBázis: <https://kaloriabazis.hu/> (Utolsó megtekintés: 2025. 09. 30.)
18. JaCoCo: <https://www.eclemma.org/jacoco/> (Utolsó megtekintés: 2026. 05. 03.)

19 Nyilatkozat

Alulírott Kanton Roderik programtervező informatikus szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztés Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök stb.) használtam fel. Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

2026.05.03.



Aláírás

20 Köszönetnyilvánítás

Szeretnék köszönetet mondani elsősorban a témavezetőmnek, Dr. Bilicki Vilmos egyetemi adjunktusnak, amiért segítette és felügyelte a munkámat.

Emellett szeretnék köszönetet mondani a családomnak, akik teljes szívükkel támogattak nemcsak a szakdolgozatom elkészítése közben, hanem az egyetemi éveim alatt is. Valamint köszönöm a megértésüket, amikor fel kellett áldoznom a velük eltöltött időt a szakdolgozatom haladása érdekében és köszönöm a segítségüket, tanácsaikat.

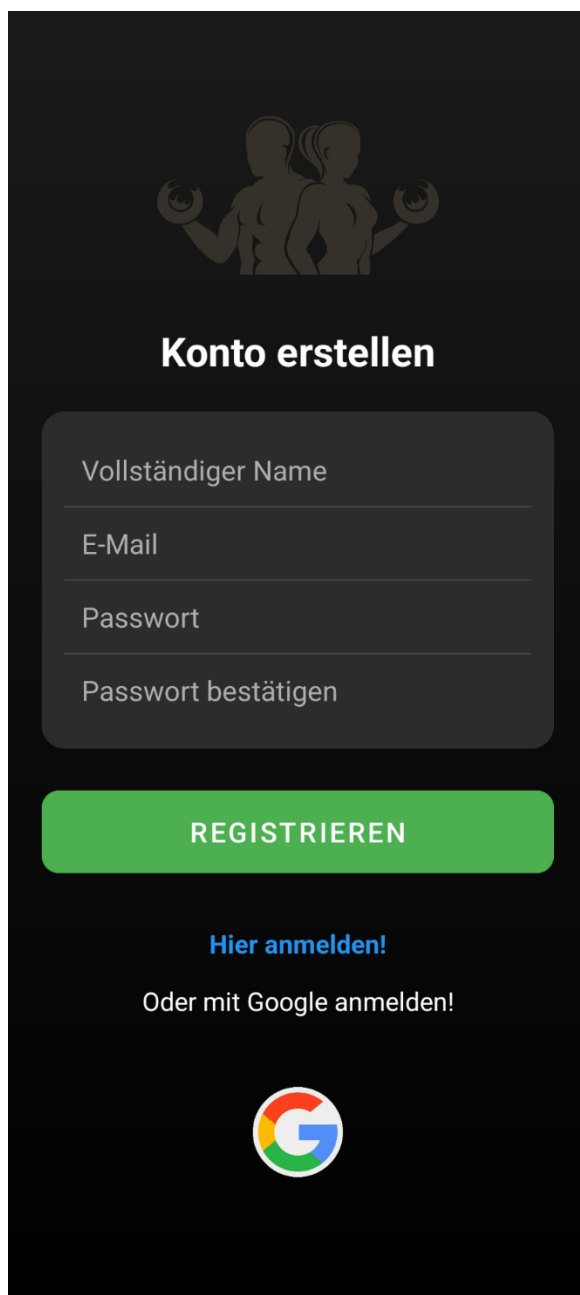
Valamint szeretnék köszönetet mondani két nagyon jó barátomnak is. Meghallgatták a panaszaimat amikor nagyon nehéz volt, segítettek és tanácsokat adtak az alkalmazással kapcsolatban felhasználói és tesztelői oldalról is.

21 Elektronikus mellékletek

1. Az alkalmazás GitHub elérhetősége:
https://github.com/KantonRoderik/Android_App (Utolsó megtekintés: 2026. 05. 03.)

22 Mellékletek

Az alkalmazás egyéb képernyőtervei, valamint a sötét témájú és német képernyőtervek: A 4 elérhető nyelvről korábban mutattam az angolt és most mutatom a német nyelven és dark mode-ot



The screenshot shows a registration screen in German with a dark background. At the top, there is a silhouette of a man and a woman holding dumbbells. Below this is the title 'Konto erstellen'. The registration form consists of four input fields: 'Vollständiger Name', 'E-Mail', 'Passwort', and 'Passwort bestätigen'. A green button labeled 'REGISTRIEREN' is positioned below the form. Underneath the button, there is a link 'Hier anmelden!' and the text 'Oder mit Google anmelden!'. At the bottom, there is a circular Google logo.

M.1. ábra: Bejelentkezés oldal (Német sötét megjelenés)



Konto erstellen

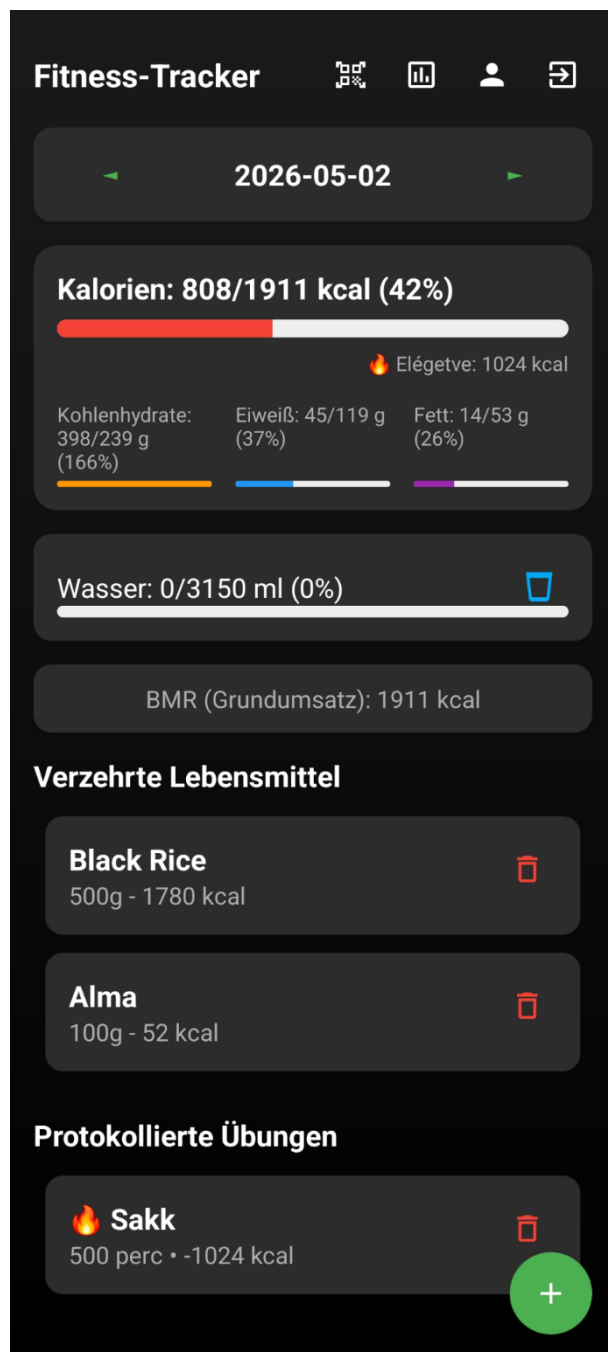
REGISTRIEREN

[Hier anmelden!](#)

Oder mit Google anmelden!



M.2. ábra: Regisztrációs oldal (Német sötét megjelenés)



M.3. ábra: Regisztrációs oldal (Német sötét megjelenés)



M.4. ábra: Profil oldal (Német sötét megjelenés)

PROFIL BEARBEITEN

PERSÖNLICHE INFO

VOLLSTÄNDIGER NAME

 kanton roderik

GESCHLECHT

Männlich

KÖRPERSTATISTIK

GEWICHT

 90 kg

GRÖSSE

 181 cm

ALTER

 25 Jahre

 **AUTO-BERECHNUNG**

SICHERHEIT

NEUES PASSWORT



PASSWORT BESTÄTIGEN



 **ÄNDERUNGEN SPEICHERN**

M.5. ábra: Adatok módosítása oldal (Német sötét megjelenés)

TAGESZIELE BEARBEITEN

Dein BMR: 1911 kcal

HASZNÁLD EZT

ERNÄHRUNGSVORLAGE

Ausgewogen

 Kalorien

1911 kcal

 Kohlenhydrate

238 g

 Fett

53 g

 Eiweiß

119 g

 Wasser

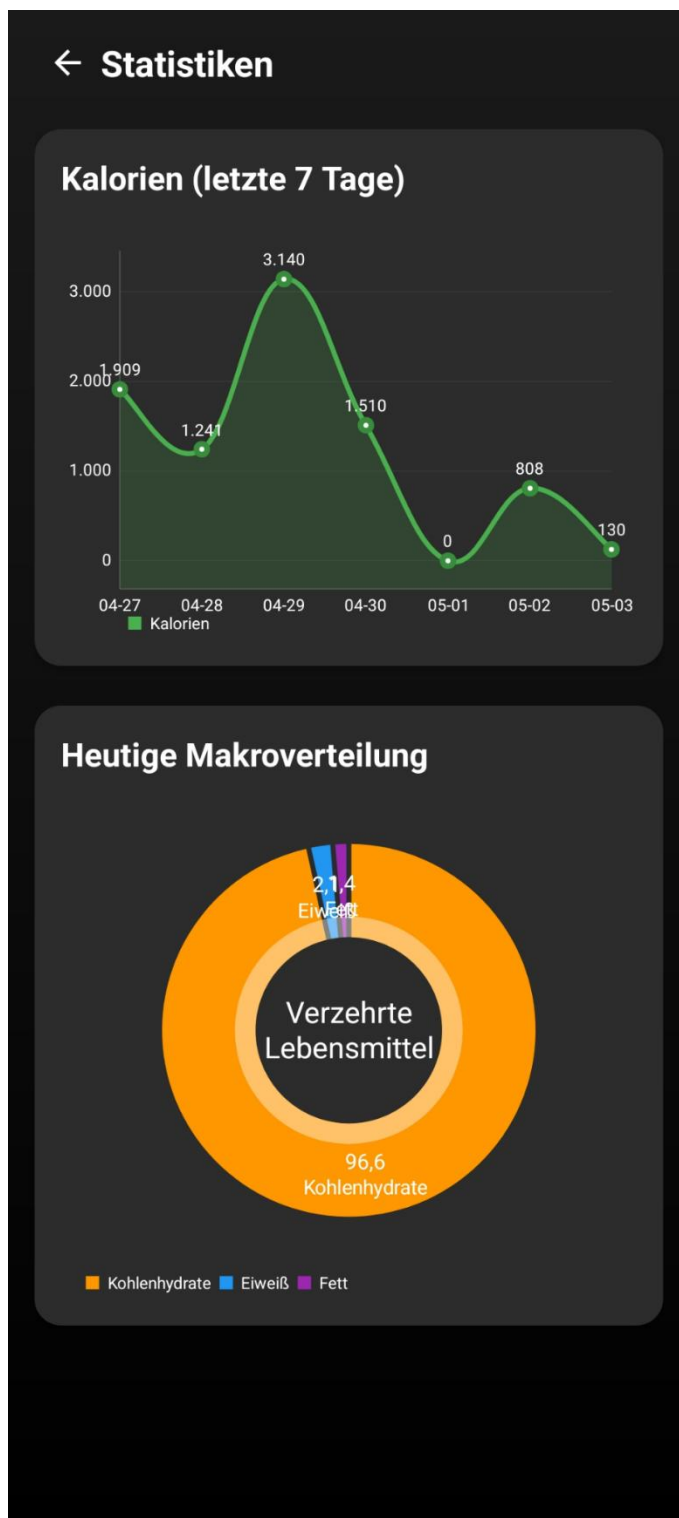
3150 ml

ÄNDERUNGEN SPEICHERN

Das Ändern dieser Werte aktualisiert die heutige und zukünftige Verfolgung.

M.6. ábra: Személyes célok módosítása oldal (Német sötét megjelenés)

Tartalmaz BMI számlálót, ami egy gombnyomásra kiszámolja a user adataiból
Illetve preset célokat (Fehérje dús, szénhidrát szegény célok)



M.7. ábra: Heti és Napi statisztika mutatása oldal (Német sötét megjelenés)

← Neues Element hinzufügen

ESSEN ÜBUNG

Pick a Food

Sült Csirkeszárny

Quantity (grams):

Enter the quantity gram ▾

ESSEN HINZUFÜGEN

M.8. ábra: Elfogyasztott étel hozzáadása oldal (Német sötét megjelenés)

A User beírja, amit evett és ha az adatbázis nem találja, akkor felugrik a lehetőség az AI által számolt adatokat használni, amit aztán beír a db-be és különböző mértékegységek is megjelennek az ételnek ami nagyban megkönnyíti a felhasználó feladatát

← **Neue Übung**

ESSEN **ÜBUNG**

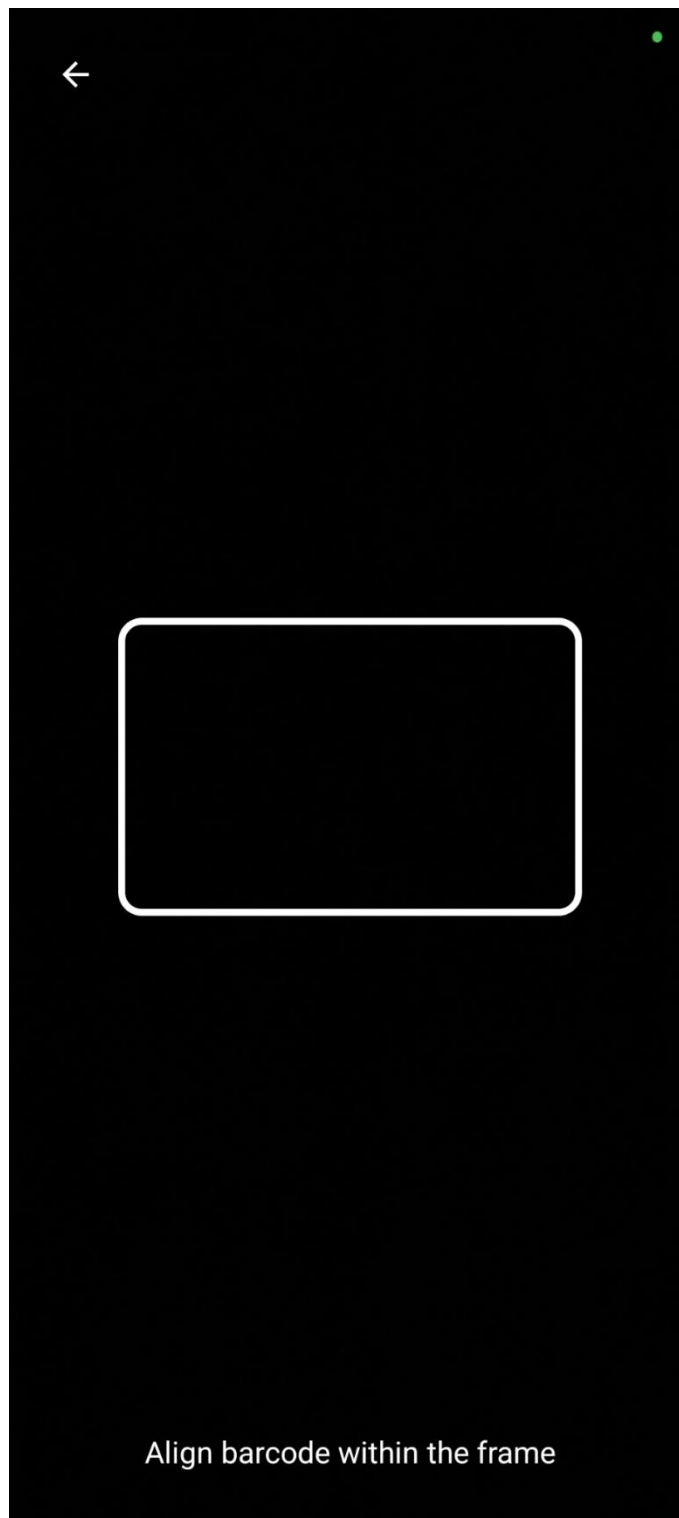
Art der Übung:
z.B. Laufen

Dauer (Min.):
0 min

[Kalorien mit KI berechnen](#)

ÜBUNG HINZUFÜGEN

M.9. ábra: Napi mozgást naplózó oldal (Német sötét megjelenés)



M.10. ábra: Barcode scanner felülete a kamera használatával (Német sötét megjelenés)